SSMRV COLLEGE
DATA ANALYTICS LAB PROGRAMS
[SEP SYLLABUS]

EXPERIMENT NO. 1

Title

Probability Distribution of Categorical Variables Using Python

Aim

To calculate the probability distribution of categorical variables such as **Gender, Batting Style, Bowling Style, Player Position, and Country** from the given dataset using Python and the Pandas library.

Problem Statement

From the given dataset **players_info.csv**, perform the following tasks:

a) Calculate the probability distribution of genders among the players.

b) Calculate the probability distribution of batting styles.

c) Calculate the probability distribution of bowling styles.

d) Calculate the probability distribution of player positions.

e) Calculate the probability distribution of countries among the players.

Learning Objectives

After completing this experiment, the student will be able to:

- Understand the concept of probability distribution.
- Read a CSV dataset using the Pandas library.
- Calculate the frequency of categorical variables.
- Compute probability values using mathematical formulas.
- Display the results in tabular format using Python.
- Verify that the sum of all probabilities equals 1.

vosuai

Theory

Data Analytics involves collecting, organizing, analysing, and interpreting data to obtain useful information. One of the fundamental statistical concepts used in data analysis is the **probability distribution**.

A probability distribution shows how the total observations in a dataset are distributed among different categories. It indicates the likelihood of occurrence of each category.

For categorical data, the probability of a category is calculated using the following formula:

$$P(E) = \frac{\text{Frequency of the Category}}{\text{Total Number of Records}}$$

Where:

- **P(E)** represents the probability of an event.
- **Frequency** is the number of times a category appears in the dataset.
- **Total Number of Records** is the total number of observations present in the dataset.

The probability values always satisfy the following condition:

$$0 \leq P(E) \leq 1$$

The sum of the probabilities of all categories is always equal to:

$$\sum P(E) = 1$$

In Python, the **Pandas** library provides the `value_counts()` function to calculate the frequency of each category, making the computation of probability distributions simple and efficient.

Software Requirements

- Operating System: Windows 10/11 or Linux
- Python 3.x
- Anaconda Distribution
- Jupyter Notebook
- Pandas Library
- CSV Dataset: **players_info.csv**

Formula

The probability of each category is calculated using:

$$Probability = \frac{Frequency}{Total\ Number\ of\ Records}$$

Percentage is calculated using:

$$Percentage = Probability \times 100$$

Algorithm

Step 1: Start Jupyter Notebook using Anaconda Navigator.

Step 2: Create a new Python Notebook.

Step 3: Import the Pandas library.

Step 4: Read the dataset `players_info.csv` using the `read_csv()` function.

Step 5: Display the first five records of the dataset.

Step 6: Display the column names of the dataset.

Step 7: Find the total number of records in the dataset.

Step 8: Calculate the frequency of each category using the `value_counts()` function.

Step 9: Calculate the probability by dividing the frequency by the total number of records.

Step 10: Convert the probability into percentage.

Step 11: Display the results in tabular format.

Step 12: Verify that the sum of all probabilities is equal to 1.

Procedure

Step 1

Open **Anaconda Navigator**.

Step 2

Launch **Jupyter Notebook**.

Step 3

Create a new **Python 3 Notebook**.

Step 4

Save the notebook with the name:

Experiment_01_Probability_Distribution.ipynb

Step 5

Copy the dataset **players_info.csv** into the same folder where the notebook is saved.

Step 6

Import the Pandas library.

import pandas as pd

Step 7

Read the CSV dataset.

df = pd.read_csv("players_info.csv")

Step 8

Display the first five records.

print(df.head())

This step verifies that the dataset has been loaded successfully.

Step 9

Display the column names.

print(df.columns)

This helps identify the available columns in the dataset.

Step 10

Find the total number of records.

total_players = len(df)

This value is used as the denominator while calculating probabilities.

Step 11

Calculate the frequency of each category using the `value_counts()` function.

Example:

gender_frequency = df["gender"].value_counts()

The `value_counts()` function counts the number of occurrences of each category.

Step 12

Calculate the probability of each category.

Example:

gender_probability = gender_frequency / total_players

The probability is calculated by dividing the frequency by the total number of records.

Step 13

Convert the probability into percentage.

percentage = gender_probability * 100

v@sugi

Step 14

Display the frequency, probability, and percentage in a tabular format.

Step 15

Repeat Steps 11–14 for the remaining categorical variables:

- Batting Style
- Bowling Style
- Player Position
- Country

Step 16

Verify the probability distribution.

print(gender_probability.sum())

The output should be: 1.0

Similarly, verify the probability distribution for all the remaining categories.

Python Program

```
# Experiment 1
# Program to calculate the probability distribution
# of Gender, Batting Style, Bowling Style,
# Position and Country.

# Import the Pandas library
import pandas as pd

# Read the dataset
df = pd.read_csv("players_info.csv")

# Display the first five records
print("First Five Records")
print(df.head())
```

@sugi

```
# Display the column names
print("\nColumn Names")
print(df.columns)

# Find the total number of players
total_players = len(df)

print("\nTotal Number of Players =", total_players)
# ---------------------------------------
# a) Probability Distribution of Gender
# ---------------------------------------
print("\na) Probability Distribution of Gender")
gender_frequency = df["gender"].value_counts()
gender_probability = gender_frequency / total_players
gender_result = pd.DataFrame({
    "Frequency": gender_frequency,
    "Probability": gender_probability,
    "Percentage": gender_probability * 100
})

print(gender_result)
print("\nTotal Probability =", gender_probability.sum())
# ---------------------------------------
# b) Probability Distribution of Batting Style
# ---------------------------------------
print("\nb) Probability Distribution of Batting Style")
batting_frequency = df["batting_style"].value_counts()
```

vcsugi

```
batting_probability = batting_frequency / total_players

batting_result = pd.DataFrame({
    "Frequency": batting_frequency,
    "Probability": batting_probability,
    "Percentage": batting_probability * 100
})
print(batting_result)
print("\nTotal Probability =", batting_probability.sum())
# ---------------------------------------
# c) Probability Distribution of Bowling Style
# ---------------------------------------
print("\nc) Probability Distribution of Bowling Style")
bowling_frequency = df["bowling_style"].value_counts()
bowling_probability = bowling_frequency / total_players

bowling_result = pd.DataFrame({
    "Frequency": bowling_frequency,
    "Probability": bowling_probability,
    "Percentage": bowling_probability * 100
})

print(bowling_result)
print("\nTotal Probability =", bowling_probability.sum())
# ---------------------------------------
# d) Probability Distribution of Position
# ---------------------------------------
print("\nd) Probability Distribution of Player Position")
```

```
position_frequency = df["position"].value_counts()
position_probability = position_frequency / total_players

position_result = pd.DataFrame({
    "Frequency": position_frequency,
    "Probability": position_probability,
    "Percentage": position_probability * 100
})
print(position_result)
print("\nTotal Probability =", position_probability.sum())
# ---------------------------------------
# e) Probability Distribution of Country
# ---------------------------------------
print("\ne) Probability Distribution of Country")
country_frequency = df["country"].value_counts()
country_probability = country_frequency / total_players
country_result = pd.DataFrame({
    "Frequency": country_frequency,
    "Probability": country_probability,
    "Percentage": country_probability * 100
})
print(country_result)
print("\nTotal Probability =",
      country_probability.sum())
```

Vosuji

Expected Output

The program should display:

- First five records of the dataset.
- Column names.
- Total number of records.
- Frequency table for each categorical variable.
- Probability table for each categorical variable.
- Percentage distribution.
- Sum of probabilities equal to 1.0.

Result

The probability distributions of Gender, Batting Style, Bowling Style, Player Position, and Country were successfully calculated from the `players_info.csv` dataset using Python and the Pandas library. The calculated probabilities were verified, and the sum of probabilities for each categorical variable was found to be equal to 1, confirming the correctness of the probability distributions.

Viva Questions

1. What is probability?
2. What is a probability distribution?
3. What is the formula for calculating probability?
4. Why is Pandas used in Python?
5. What is a DataFrame?
6. What is the use of value_counts()?
7. Why is read_csv() used?
8. What does len(df) return?
9. Why should the sum of all probabilities be equal to 1?
10. Explain the output of your program.

OUTPUT:

```
First Five Records
  player_id   player_name gender batting_style   bowling_style      position  \
0      P001  Aarav Sharma   Male        Switch  Right-arm pace        Batter
1      P002   Rohan Singh   Male    Right-hand  Right-arm pace        Bowler
2      P003  Grace Sharma   Male     Left-hand  Right-arm spin        Batter
3      P004  Arjun Sharma   Male     Left-hand  Right-arm spin  Wicketkeeper
4      P005   Kabir Brown   Male    Right-hand  Right-arm pace        Batter

     country
0   Pakistan
1      India
2  Australia
3  Australia
4  Australia

Column Names
Index(['player_id', 'player_name', 'gender', 'batting_style', 'bowling_style',
       'position', 'country'],
      dtype='object')

Total Number of Players = 80

a) Probability Distribution of Gender
        Frequency  Probability  Percentage
gender
Male           56          0.7        70.0
Female         24          0.3        30.0

Total Probability = 1.0
```

```
b) Probability Distribution of Batting Style
               Frequency  Probability  Percentage
batting_style
Right-hand            48         0.60        60.0
Left-hand             28         0.35        35.0
Switch                 4         0.05         5.0

Total Probability = 1.0

c) Probability Distribution of Bowling Style
                    Frequency  Probability  Percentage
bowling_style
Right-arm pace             24        0.300        30.0
Right-arm spin             20        0.250        25.0
Left-arm pace              14        0.175        17.5
Left-arm spin              12        0.150        15.0
Wicketkeeper/None          10        0.125        12.5

Total Probability = 1.0

d) Probability Distribution of Player Position
              Frequency  Probability  Percentage
position
Batter               24        0.300        30.0
Bowler               22        0.275        27.5
All-rounder          20        0.250        25.0
Wicketkeeper         14        0.175        17.5

Total Probability = 1.0
```

```
e) Probability Distribution of Country
             Frequency  Probability  Percentage
country
India               20       0.2500       25.00
Australia           12       0.1500       15.00
England             12       0.1500       15.00
South Africa        10       0.1250       12.50
New Zealand          8       0.1000       10.00
West Indies          7       0.0875        8.75
Pakistan             6       0.0750        7.50
Sri Lanka            5       0.0625        6.25

Total Probability = 1.0
```

EXPERIMENT NO. 2

Title

Binomial Probability Distribution Using Python

Aim

To calculate the probability of different outcomes using the **Binomial Probability Distribution** and visualize the probability distribution using Python.

Problem Statement

80% of all the visitors to **Museum of Goa** end up buying souvenirs from the souvenir shop at the museum.

On the coming Sunday, if a random sample of **10 visitors** is selected, determine:

a) The probability that **every visitor** will buy a souvenir.

b) The probability that **at most (maximum) 7 visitors** will buy souvenirs.

Also display the complete probability distribution and plot the Binomial Distribution graph.

vasugi

Learning Objectives

After completing this experiment, the student will be able to:

- Understand Binomial Probability Distribution.
- Calculate probabilities using the Binomial Formula.
- Apply mathematical concepts using Python.
- Display probability distributions in tabular form.
- Visualize probability distributions using a bar chart.
- Interpret probability values.

Theory

The **Binomial Distribution** is a discrete probability distribution that describes the probability of obtaining exactly **x successes** in **n independent trials**, where each trial has only two possible outcomes:

- Success
- Failure

The probability of success remains constant for every trial.

A Binomial Distribution is applicable when:

- There are a fixed number of trials.
- Each trial has only two outcomes.
- The probability of success remains constant.
- Each trial is independent.

In this experiment,

- Success = Visitor buys a souvenir.
- Failure = Visitor does not buy a souvenir.

Since every visitor either buys or does not buy a souvenir, the problem follows a **Binomial Distribution**.

Software Requirements

- Windows 10/11
- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas Library
- Matplotlib Library

Algorithm

Step 1: Start Jupyter Notebook.

Step 2: Import the required Python libraries.

Step 3: Initialize the values of **n**, **p**, and **q**.

Step 4: Calculate the probability that all visitors buy souvenirs.

Step 5: Calculate probabilities for 0 to 7 visitors.

Step 6: Add these probabilities to obtain the probability of at most 7 visitors buying souvenirs.

Step 7: Calculate the probabilities for all values from 0 to 10.

Step 8: Store the probabilities in a DataFrame.

Step 9: Display the probability distribution.

Step 10: Plot the Binomial Distribution graph.

Procedure

Step 1

Open **Anaconda Navigator**.

Step 2

Launch **Jupyter Notebook**.

Step 3

Create a new Python Notebook.

Step 4

Import the required libraries.

```
from math import comb
import pandas as pd
import matplotlib.pyplot as plt
```

v@suji

Step 5

Assign the values.

n=10

p=0.80

q=0.20

Step 6

Calculate the probability that every visitor buys a souvenir.

Step 7

Calculate the probability that at most seven visitors buy souvenirs.

Step 8

Prepare the probability distribution table.

Step 9

Display the probability distribution.

Step 10

Plot the Binomial Distribution graph.

Python Program

```
# Experiment 2
# Binomial Probability Distribution

from math import comb
import pandas as pd
import matplotlib.pyplot as plt

# Number of visitors
n = 10

# Probability of buying a souvenir
p = 0.80
```

v@suji

```
# Probability of not buying
q = 1 - p

# ------------------------------------
# Question 1
# Probability that every visitor buys
# ------------------------------------

x = 10

prob_everyone = comb(n, x) * (p ** x) * (q ** (n - x))

print("QUESTION 1")
print("Probability that every visitor buys a souvenir =", prob_everyone)
print("Percentage =", prob_everyone * 100)

# ------------------------------------
# Question 2
# Probability that at most 7 visitors buy
# ------------------------------------

prob_max_7 = 0

probability_values = []

for x in range(11):

    probability = comb(n, x) * (p ** x) * (q ** (n - x))

    probability_values.append(probability)

    if x <= 7:
        prob_max_7 += probability

print("\nQUESTION 2")
print("Probability that at most 7 visitors buy =", prob_max_7)
print("Percentage =", prob_max_7 * 100)
```

v@sugi

```
# ------------------------------------
# Probability Distribution Table
# ------------------------------------

distribution = pd.DataFrame({ "Visitors Buying": range(11), "Probability":
probability_values })

print("\nComplete Probability Distribution")

print(distribution)

print("\nTotal Probability =", sum(probability_values))

# ------------------------------------
# Final Answers
# ------------------------------------

print("\nFINAL ANSWERS")

print("P(X=10) =", round(prob_everyone,4))

print("P(X<=7) =", round(prob_max_7,4))

# ------------------------------------
# Graph
# ------------------------------------

plt.figure(figsize=(8,5))

plt.bar(range(11), probability_values)

plt.xlabel("Number of Visitors Buying Souvenirs")

plt.ylabel("Probability")

plt.title("Binomial Probability Distribution")

plt.xticks(range(11))

plt.grid(axis="y")

plt.show()
```

v@sugi

Explanation of the Program

`comb(n,x)`

Calculates the binomial coefficient

$$\binom{n}{x}$$

`p**x`

Calculates

$$p^x$$

`q**(n-x)`

Calculates

$$q^{(n-x)}$$

`probability_values.append(probability)`

Stores every probability from **0 to 10**.

`pd.DataFrame()`

Displays the probability distribution in table format.

`plt.bar()`

Draws the Binomial Distribution graph.

Expected Output

```
QUESTION 1

Probability that every visitor buys a souvenir = 0.1073741824

Percentage = 10.73741824
```

```
QUESTION 2

Probability that at most 7 visitors buy = 0.3222004736

Percentage = 32.22004736
```

```
FINAL ANSWERS

P(X=10)=0.1074

P(X<=7)=0.3222
```

The program will also display:

- Complete probability distribution table.
- Binomial Distribution bar chart.

Interpretation

- The probability that **all 10 visitors** buy souvenirs is approximately **0.1074 (10.74%)**.
- The probability that **at most 7 visitors** buy souvenirs is approximately **0.3222 (32.22%)**.
- The probability distribution table shows the probability for every possible number of visitors (0–10) buying souvenirs.
- The bar chart visually represents the Binomial Distribution, with the highest probability occurring around the expected number of buyers.

Result

The Binomial Probability Distribution was successfully used to calculate the probability that every visitor buys a souvenir and the probability that at most seven visitors buy souvenirs. The probability distribution was displayed in tabular form and visualized using a bar chart.

OUTPUT:

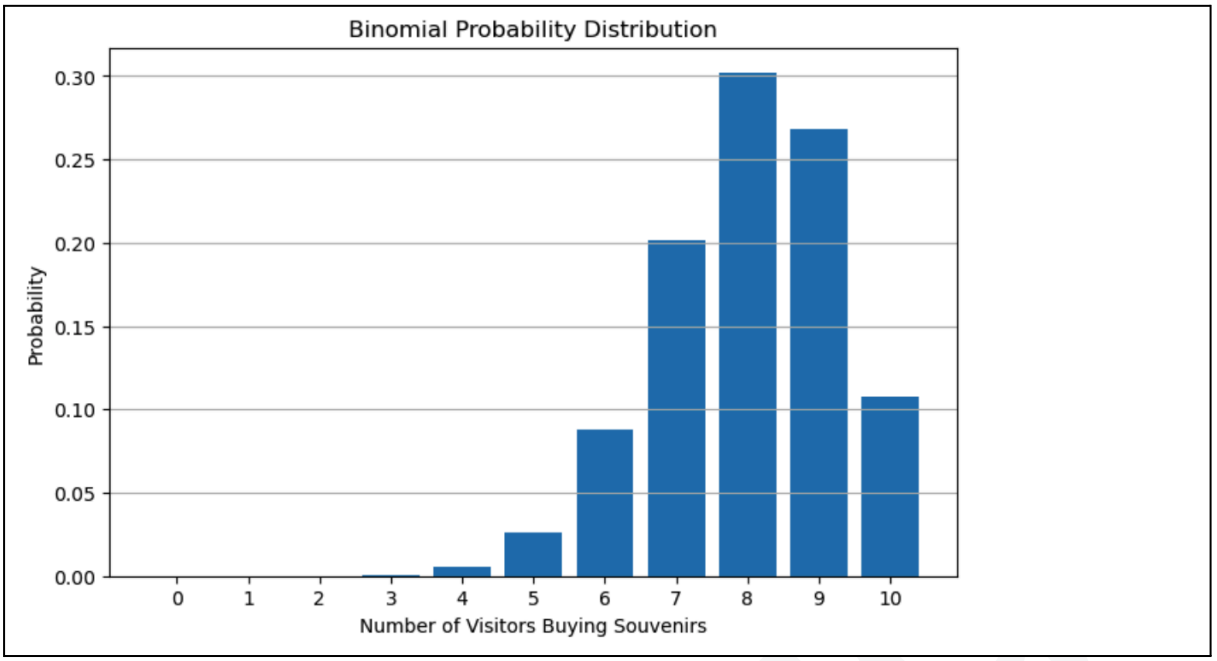
```
QUESTION 1
Probability that every visitor buys a souvenir = 0.10737418240000006
Percentage = 10.737418240000006

QUESTION 2
Probability that at most 7 visitors buy = 0.32220047359999987
Percentage = 32.22004735999999

Complete Probability Distribution
    Visitors Buying   Probability
0                 0  1.024000e-07
1                 1  4.096000e-06
2                 2  7.372800e-05
3                 3  7.864320e-04
4                 4  5.505024e-03
5                 5  2.642412e-02
6                 6  8.808038e-02
7                 7  2.013266e-01
8                 8  3.019899e-01
9                 9  2.684355e-01
10               10  1.073742e-01

Total Probability = 1.0

FINAL ANSWERS
P(X=10) = 0.1074
P(X<=7) = 0.3222
```



Viva Questions

1. What is a Binomial Distribution?
2. What are the conditions for a Binomial experiment?
3. What is meant by a Bernoulli trial?
4. What is the difference between success and failure?
5. What is the value of **q** if **p = 0.8**?
6. What does `comb()` calculate?
7. Why is `range(11)` used?
8. Why should the total probability equal 1?
9. Why is a bar graph suitable for this distribution?
10. Give two real-life applications of the Binomial Distribution.

EXPERIMENT NO. 3

Title

Probability Distribution of SAT Scores Using Python

Aim

To calculate the probability that a student scores **less than 800** and **more than 1300** in the SAT examination using Python.

Problem Statement

A testing agency wants to analyze the complexity of the **SAT Exam 2022**. They have collected the SAT scores of **1000 students** in the file **sat_score.csv**.

Determine:

a) The probability that a student scores **less than 800**.

b) The probability that a student scores **more than 1300**.

Learning Objectives

After completing this experiment, the student will be able to:

- Read a CSV dataset using Python.
- Calculate probabilities from real-world data.
- Understand frequency and probability.
- Display probability values.
- Visualize the data using a histogram.

Theory

Probability is the measure of the likelihood of an event occurring.

For a dataset,

$$P(E) = \frac{\text{Number of favourable observations}}{\text{Total number of observations}}$$

where

- **Favourable observations** = Number of students satisfying the condition.
- **Total observations** = Total number of students.

In this experiment,

- Event 1 = Student scores less than 800.
- Event 2 = Student scores more than 1300.

The probability is obtained by counting the number of students satisfying the condition and dividing it by the total number of students.

Formula

Probability Formula

$$P(E) = \frac{\text{Frequency of Event}}{\text{Total Number of Records}}$$

For Question (a)

$$P(SAT < 800) = \frac{\text{Students scoring below 800}}{1000}$$

For Question (b)

$$P(SAT > 1300) = \frac{\text{Students scoring above 1300}}{1000}$$

Software Requirements

- Windows 10/11
- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas
- Matplotlib
- sat_score.csv

Algorithm

Step 1: Open Jupyter Notebook.

Step 2: Import the required libraries.

Step 3: Read the dataset.

Step 4: Display the first five records.

Step 5: Find the total number of students.

Step 6: Count the number of students scoring below 800.

Step 7: Count the number of students scoring above 1300.

Step 8: Calculate the probabilities.

Step 9: Display the probabilities.

Step 10: Plot the distribution of SAT scores.

Procedure

Step 1

Open **Anaconda Navigator**.

Step 2

Launch **Jupyter Notebook**.

Step 3

Create a new Python Notebook.

Step 4

Import the required libraries.

```
import pandas as pd

import matplotlib.pyplot as plt
```

Step 5

Read the dataset.

```
df = pd.read_csv("sat_score.csv")
```

v@sugi

Step 6

Display the first five records.

print(df.head())

Step 7

Display the column names.

print(df.columns)

Step 8

Find the total number of students.

total_students = len(df)

Step 9

Count the students scoring below 800.

less_800 = len(df[df["SAT_score"] < 800])

Step 10

Count the students scoring above 1300.

more_1300 = len(df[df["SAT_score"] > 1300])

Step 11

Calculate the probabilities.

prob_less_800 = less_800 / total_students

prob_more_1300 = more_1300 / total_students

Step 12

Display the results.

Step 13

Draw the histogram.

v@sugi

Python Program

Probability of SAT Scores

```
import pandas as pd
import matplotlib.pyplot as plt

# Read dataset
df = pd.read_csv("sat_score.csv")

# Display first five records
print("First Five Records")
print(df.head())
# Display column names
print("\nColumn Names")
print(df.columns)

# Total number of students
total_students = len(df)
print("\nTotal Students =", total_students)

# Students scoring below 800
less_800 = len(df[df["SAT_score"] < 800])

# Students scoring above 1300
more_1300 = len(df[df["SAT_score"] > 1300])

# Calculate probabilities
prob_less_800 = less_800 / total_students
prob_more_1300 = more_1300 / total_students
```

v@sugi

Display results

print("\nStudents scoring below 800 =", less_800)

print("Probability =", prob_less_800)

print("Percentage =", prob_less_800 * 100)

print("\nStudents scoring above 1300 =", more_1300)

print("Probability =", prob_more_1300)

print("Percentage =", prob_more_1300 * 100)

Histogram

plt.figure(figsize=(8,5))

plt.hist(df["SAT_score"], bins=20)

plt.title("Distribution of SAT Scores")

plt.xlabel("SAT Score")

plt.ylabel("Number of Students")

plt.grid(True)

plt.show()

Explanation of the Program

- `pd.read_csv()` reads the dataset.
- `len(df)` calculates the total number of students.
- `df["SAT_score"] < 800` filters students scoring below 800.
- `df["SAT_score"] > 1300` filters students scoring above 1300.
- Probability is calculated by dividing the number of favourable students by the total number of students.
- `plt.hist()` creates a histogram to visualize the score distribution.

@sugi

OUTPUT:

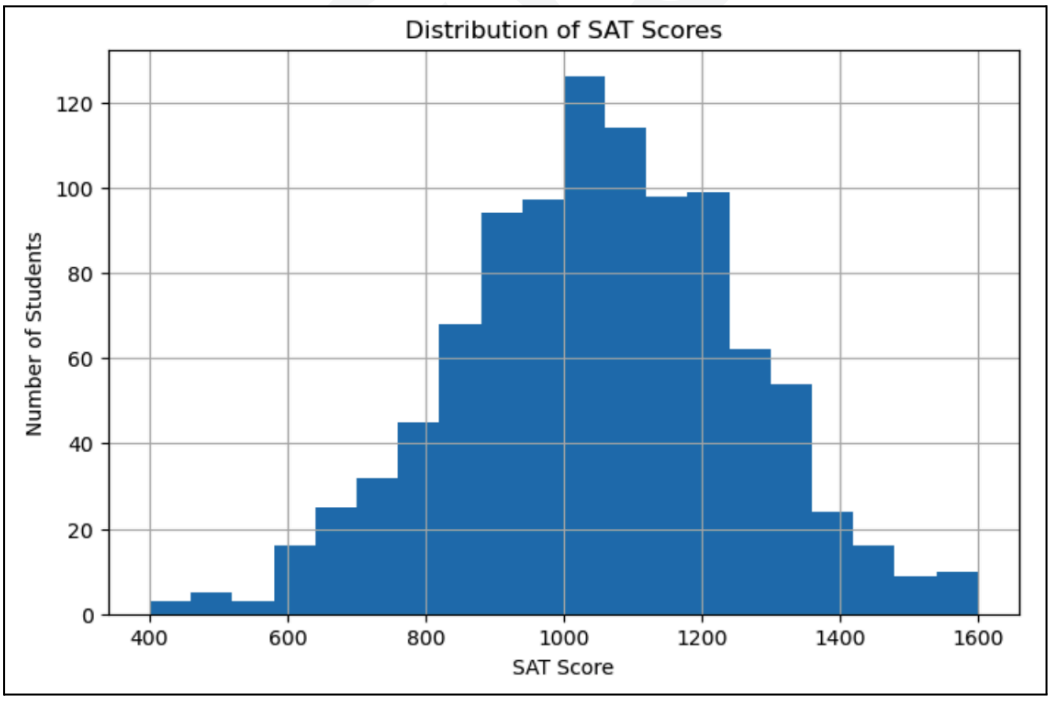
```
First Five Records
  student_id  SAT_score
0      S0001       1037
1      S0002        920
2      S0003        682
3      S0004       1188
4      S0005       1129

Column Names
Index(['student_id', 'SAT_score'], dtype='object')

Total Students = 1000

Students scoring below 800 = 109
Probability = 0.109
Percentage = 10.9

Students scoring above 1300 = 112
Probability = 0.112
Percentage = 11.200000000000001
```



Interpretation

- Out of **1000 students**, **109 students** scored **less than 800**.
- Therefore,

$$P(SAT < 800) = 0.109$$

- Out of **1000 students**, **112 students** scored **more than 1300**.
- Therefore,

$$P(SAT > 1300) = 0.112$$

These probabilities indicate that approximately **10.9%** of the students scored below 800, while **11.2%** scored above 1300.

Result

The probabilities of students scoring **less than 800** and **more than 1300** in the SAT examination were successfully calculated from the given dataset using Python. The results show that the probability of scoring below 800 is **0.109 (10.9%)**, and the probability of scoring above 1300 is **0.112 (11.2%)**. The distribution of SAT scores was also visualized using a histogram.

Viva Questions

1. What is probability?
2. What is a probability distribution?
3. What is a CSV file?
4. What is the purpose of `read_csv()`?
5. What does `len(df)` return?
6. How do you filter records based on a condition in Pandas?
7. How is the probability of an event calculated from a dataset?
8. What does `plt.hist()` do?
9. Why is a histogram used for SAT scores?
10. What is the probability of a student scoring less than 800 in this dataset?
11. What is the probability of a student scoring more than 1300 in this dataset?
12. What conclusion can you draw from the calculated probabilities?

EXPERIMENT NO. 4

Title

One-Sample t-Test for Mean Mobile Internet Usage Time

Aim

To determine whether there is significant evidence that the population mean time spent per day accessing the Internet through a mobile device is **different from 144 minutes**, using a **One-Sample t-Test** at a **5% level of significance**.

Problem Statement

A Marketing services company reported that the typical American spends a mean of **144 minutes (2.4 hours) per day** accessing the Internet via a mobile device.

A sample of **30 friends and family members** was selected, and their mobile Internet access times are stored in the CSV file `MobileTime.csv`.

Determine whether there is evidence that the **population mean time** spent per day accessing the Internet via a mobile device is **different from 144 minutes**.

Use: $\alpha=0.05$

Learning Objectives

After completing this experiment, the student will be able to:

- Understand the concept of hypothesis testing.
- Understand the One-Sample t-Test.
- Formulate null and alternative hypotheses.
- Calculate a sample mean and sample standard deviation.
- Calculate the t-statistic.
- Interpret the p-value.
- Make a statistical decision based on the level of significance.
- Draw a conclusion from the hypothesis test using Python.

Theory

A **hypothesis test** is a statistical method used to determine whether there is sufficient evidence in a sample to support a claim about a population.

In this experiment, the company claims that the mean mobile Internet usage time is: μ=144

We have a sample of 30 observations and want to determine whether the actual population mean is **different from 144 minutes**.

Since:

- There is only **one sample**.
- The population mean to be tested is **144 minutes**.
- The population standard deviation is not given.
- The sample size is 30.

We use a **One-Sample t-Test**.

Hypotheses

Since the question asks whether the population mean is **different from 144 minutes**, this is a **two-tailed test**.

Null Hypothesis

$$H_0 : \mu = 144$$

The population mean mobile Internet usage time is 144 minutes.

Alternative Hypothesis

$$H_1 : \mu \neq 144$$

The population mean mobile Internet usage time is different from 144 minutes.

Level of Significance

The given level of significance is:

$$\alpha = 0.05$$

Therefore, the test will be conducted at the **5% significance level**.

Formula

The One-Sample t-Test statistic is calculated using:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

where:

- $\bar{x}$ = Sample mean
- μ_0 = Hypothesized population mean
- s = Sample standard deviation
- n = Sample size

In this experiment:

$$\mu_0 = 144$$

and

$$n = 30$$

Decision Rule

The decision is based on the **p-value**.

If:

$$p\text{-value} < \alpha$$

then **Reject** H_0.

If:

$$p\text{-value} \geq \alpha$$

then **Fail to Reject** H_0.

Here:

$$\alpha = 0.05$$

Software Requirements

- Python 3.x
- Anaconda Distribution
- Jupyter Notebook
- Pandas Library
- SciPy Library
- Matplotlib Library
- `MobileTime.csv`

Dataset Description

The supplied dataset contains **30 observations**.

Column	Description
`participant_id`	Unique identification number of each participant
`MobileTime`	Time spent accessing the Internet through a mobile device, in minutes

The dataset contains:

$$n = 30$$

observations.

Algorithm

Step 1: Start Jupyter Notebook.

Step 2: Import the required Python libraries.

Step 3: Read the InternetMobileTime.csv dataset.

Step 4: Display the first five records.

Step 5: Display the column names.

Step 6: Find the total number of observations.

Step 7: Calculate the sample mean of MobileTime.

Step 8: Perform a One-Sample t-Test by comparing the sample mean with 144 minutes.

Step 9: Obtain the t-statistic and p-value.

Step 10: Compare the p-value with the significance level α=0.05.

Step 11: Reject or fail to reject the null hypothesis.

Step 12: Interpret the result and state the conclusion.

Procedure

Step 1: Open Anaconda Navigator

Open **Anaconda Navigator** from your computer.

Step 2: Launch Jupyter Notebook

Open **Jupyter Notebook**.

Step 3: Create a New Notebook

Create a new Python notebook.

Save the notebook as: Experiment_04_One_Sample_t_Test.ipynb

Step 4: Place the Dataset

Place the file: MobileTime.csv

in the same folder as the Jupyter Notebook.

Step 5: Import the Required Libraries

Enter: import pandas as pd

from scipy.stats import ttest_1samp

Step 6: Read the Dataset

df = pd.read_csv("InternetMobileTime.csv")

Step 7: Display the Dataset

print(df.head())

This displays the first five observations.

Step 8: Check the Column Names

print(df.columns)

The dataset should display:

Index(['participant_id', 'MobileTime'], dtype='object')

vtosugi

Step 9: Find the Number of Observations

```
n = len(df)
```

The result is: 30

Step 10: Calculate the Sample Mean

```
sample_mean = df["MobileTime"].mean()
```

For the supplied dataset: $\bar{x}=156.1333$ minutes.

Step 11: Perform the One-Sample t-Test

The hypothesized mean is 144 minutes.

```
t_statistic, p_value = ttest_1samp(df["MobileTime"], 144)
```

Step 12: Compare the p-value with 0.05

If the p-value is less than 0.05, reject the null hypothesis.

v@sugi

Python Program

```
# Experiment 4
# One-Sample t-Test

import pandas as pd
from scipy.stats import ttest_1samp

# Read the dataset
df = pd.read_csv("InternetMobileTime.csv")

# Display first five records
print("First Five Records")
print(df.head())

# Display column names
print("\nColumn Names")
print(df.columns)

# Find total number of observations
n = len(df)

print("\nTotal Number of Observations =", n)

# Calculate sample mean
sample_mean = df["MobileTime"].mean()

print("\nSample Mean =", sample_mean)

# Hypothesized population mean
population_mean = 144

# Perform One-Sample t-Test
t_statistic, p_value = ttest_1samp(
    df["MobileTime"],
    population_mean
)

# Display test results
print("\nOne-Sample t-Test Results")

print("t-statistic =", t_statistic)

print("p-value =", p_value)
```

vtosugi

```
# Level of significance
alpha = 0.05

print("\nLevel of Significance =", alpha)

# Decision
if p_value < alpha:

    print("\nReject the Null Hypothesis")

    print("Conclusion: There is sufficient evidence that")
    print("the population mean mobile Internet usage time")
    print("is different from 144 minutes.")

else:

    print("\nFail to Reject the Null Hypothesis")

    print("Conclusion: There is not sufficient evidence that")
    print("the population mean mobile Internet usage time")
    print("is different from 144 minutes.")
```

Expected Output

For your supplied dataset, the important output will be approximately:

First Five Records

	participant_id	MobileTime
0	M01	132
1	M02	148
2	M03	151
3	M04	167
4	M05	143

Total Number of Observations = 30

Sample Mean = 156.13333333333333

One-Sample t-Test Results

t-statistic = 5.812140901078351

p-value = 2.6699618932126e-06

Level of Significance = 0.05

Reject the Null Hypothesis

Conclusion: There is sufficient evidence that the population mean mobile Internet usage time is different from 144 minutes.

Explanation of the Python Program

1. Import Pandas

import pandas as pd

Pandas is used to read and work with the CSV dataset.

2. Import the One-Sample t-Test

from scipy.stats import ttest_1samp

`ttest_1samp()` is the SciPy function used to perform a One-Sample t-Test.

3. Read the CSV File

df = pd.read_csv("InternetMobileTime.csv")

This reads the dataset and stores it in the DataFrame `df`.

4. Find the Number of Observations

n = len(df)

`len(df)` returns the number of rows in the dataset. For this dataset: n=30

5. Calculate the Sample Mean

sample_mean = df["MobileTime"].mean()

The `.mean()` function calculates the average mobile Internet usage time.

For the given dataset: $\bar{x}$=156.1333 minutes.

6. Specify the Hypothesized Mean

population_mean = 144

The value 144 comes from the problem statement.

7. Perform the t-Test

t_statistic, p_value = ttest_1samp(df["MobileTime"], population_mean)

This compares the sample mean with the hypothesized population mean of 144 minutes.

The function returns two values:

- **t-statistic**
- **p-value**

8. Set the Significance Level

alpha = 0.05

The problem specifies a significance level of 5%.

9. Make the Decision

if p_value < alpha:

If the p-value is less than 0.05, the null hypothesis is rejected.

Interpretation of the Output

The calculated values are:

$$t = 5.8121$$

and

$$p = 2.67 \times 10^{-6}$$

or approximately:

$$p = 0.00000267$$

Compare the p-value with the significance level:

$$0.00000267 < 0.05$$

Therefore, we **reject the null hypothesis**.

The sample mean is:

$$156.13 \text{ minutes}$$

which is higher than the reported mean of 144 minutes.

Statistical Conclusion

Since the p-value is less than 0.05, there is **statistically significant evidence** that the population mean time spent accessing the Internet through a mobile device is **different from 144 minutes per day**.

Therefore:

$$\boxed{\text{Reject } H_0}$$

and

$$\boxed{\text{Accept } H_1}$$

in the practical sense that the data provide evidence for the alternative hypothesis.

Result

The **One-Sample t-Test** was successfully performed on the `MobileTime.csv` dataset.

At the **5% level of significance**, the calculated p-value is approximately **0.00000267**, which is less than 0.05. Hence, the null hypothesis is rejected.

Therefore, there is **significant evidence that the population mean time spent accessing the Internet via a mobile device is different from 144 minutes per day**.

Viva Questions – Experiment 4

Basic Questions

1. What is hypothesis testing?
2. What is a One-Sample t-Test?
3. Why is a One-Sample t-Test used in this experiment?
4. What is the population mean given in the problem?
5. What is the sample size?
6. What is the sample mean obtained from the dataset?
7. What is the level of significance?
8. What is the null hypothesis?
9. What is the alternative hypothesis?
10. Is this a one-tailed or two-tailed test? Why?

Formula and Statistics

11. What is the formula for the One-Sample t-Test?
12. What is a t-statistic?
13. What is a p-value?
14. What are the degrees of freedom in this experiment?
15. How are degrees of freedom calculated?
16. What does the significance level α=0.05 mean?

Python Questions

17. Which library is used to perform the t-Test?
18. What is the purpose of `ttest_1samp()`?
19. What does the following statement do? df["MobileTime"].mean()
20. What does `len(df)` return?
21. Why is `144` passed to `ttest_1samp()`?
22. What does the `p_value` represent?
23. What happens if the p-value is less than 0.05?
24. What happens if the p-value is greater than 0.05?

Dataset/Result Questions

25. What is the sample mean in this experiment?
26. What is the calculated t-statistic?
27. What is the calculated p-value?
28. Is the p-value less than 0.05?
29. Should the null hypothesis be rejected?
30. What is your final conclusion from the experiment?

Most Important Viva Question

Why is this called a two-tailed test?

Answer: The question asks whether the population mean is **"different from" 144 minutes**. It does not specifically ask whether it is greater than or less than 144. Therefore, both directions are considered: H1:μ=144 Hence, it is a **two-tailed One-Sample t-Test**.

vtusuji

EXPERIMENT NO. 5

Title

Comparison of Mean Luggage Delivery Time Between Two Hotel Wings Using Independent Two-Sample t-Test

Aim

To determine whether there is a significant difference between the **mean luggage delivery time of Wing A and Wing B** of a hotel using an **Independent Two-Sample t-Test** at a **5% level of significance**.

Problem Statement

A hotel manager wants to improve the initial impression that hotel guests have when they check in. One factor contributing to the initial impression is the time taken to deliver a guest's luggage to the room after check-in.

A random sample of **20 deliveries** from **Wing A** and a random sample of **20 deliveries** from **Wing B** are selected. The results are stored in `Luggage.csv`.

Analyze the data and determine whether there is a difference between the **mean delivery time in the two wings**.

Use: $\alpha=0.05$

Learning Objectives

After completing this experiment, students will be able to:

- Understand comparison of two population means.
- Understand the Independent Two-Sample t-Test.
- Formulate null and alternative hypotheses.
- Calculate the mean delivery time for two groups.
- Calculate and interpret the t-statistic and p-value.
- Make a statistical decision using the significance level.
- Draw a conclusion from the test results using Python.

Theory

In this experiment, we have **two independent groups**:

- Wing A
- Wing B

We want to determine whether their mean luggage delivery times are significantly different.

Therefore, an **Independent Two-Sample t-Test** is used.

The test compares the means of two independent samples.

Let: μA=Mean delivery time of Wing A

μB=Mean delivery time of Wing B

Hypotheses

Since the question asks whether there is a **difference** between the two wings, this is a **two-tailed test**.

Null Hypothesis

$$H_0 : \mu_A = \mu_B$$

There is no significant difference between the mean delivery times of Wing A and Wing B.

Alternative Hypothesis

$$H_1 : \mu_A \neq \mu_B$$

There is a significant difference between the mean delivery times of Wing A and Wing B.

Level of Significance

Given:

$$\alpha = 0.05$$

Therefore, the test is conducted at the **5% level of significance**.

Formula

For two independent samples, the t-statistic can be expressed as:

$$t = \frac{\bar{X}_A - \bar{X}_B}{\sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}}}$$

where:

- $\bar{X}_A$ = Mean of Wing A
- $\bar{X}_B$ = Mean of Wing B
- s_A = Standard deviation of Wing A
- s_B = Standard deviation of Wing B
- n_A = Number of observations in Wing A
- n_B = Number of observations in Wing B

For this dataset:

$$n_A = 20$$

$$n_B = 20$$

Decision Rule

Compare the p-value with:

$$\alpha = 0.05$$

If:

$$p < 0.05$$

Reject H_0.

If:

$$p \geq 0.05$$

Fail to Reject H_0.

Software Requirements

- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas
- SciPy
- Luggage.csv

Dataset Description

The attached dataset contains three columns:

Column	Description
delivery_id	Unique identification number of each delivery
Wing	Hotel wing – A or B
DeliveryTime	Time taken to deliver luggage

There are **40 observations in total**:

- Wing A = 20 observations
- Wing B = 20 observations

The delivery time is measured in minutes.

Algorithm

Step 1: Open Jupyter Notebook.

Step 2: Import Pandas and the required statistical function from SciPy.

Step 3: Read the Luggage.csv dataset.

Step 4: Display the first five records.

Step 5: Display the column names.

Step 6: Separate the delivery times of Wing A and Wing B.

Step 7: Calculate the mean delivery time of Wing A.

Step 8: Calculate the mean delivery time of Wing B.

Step 9: Perform the Independent Two-Sample t-Test.

Step 10: Obtain the t-statistic and p-value.

Step 11: Compare the p-value with 0.05.

Step 12: Make the statistical decision.

Step 13: State the final conclusion.

Procedure

Step 1: Open Jupyter Notebook

Open **Anaconda Navigator** and launch **Jupyter Notebook**.

Step 2: Create a New Notebook

Create a new Python notebook.

Step 3: Place the Dataset

Place: Luggage.csv in the same folder as the Jupyter Notebook.

vosuji

Step 4: Import Libraries

Enter: import pandas as pd from scipy.stats import ttest_ind

Step 5: Read the Dataset

```
df = pd.read_csv("Luggage.csv")
```

Step 6: Display the First Five Records

```
print("First Five Records")

print(df.head())
```

Step 7: Display Column Names

```
print("\nColumn Names")

print(df.columns)
```

Step 8: Separate Wing A Data

```
wing_a = df[df["Wing"] == "A"]["DeliveryTime"]
```

This selects only the delivery times belonging to Wing A.

Step 9: Separate Wing B Data

```
wing_b = df[df["Wing"] == "B"]["DeliveryTime"]
```

This selects only the delivery times belonging to Wing B.

Step 10: Calculate the Means

```
mean_a = wing_a.mean()

mean_b = wing_b.mean()
```

Step 11: Perform the t-Test

```
t_statistic, p_value = ttest_ind(wing_a, wing_b)
```

Step 12: Set the Significance Level

```
alpha = 0.05
```

Step 13: Make the Decision

If the p-value is less than 0.05, reject the null hypothesis.

Python Program

```
# Experiment 5
# Comparison of Luggage Delivery Time
# between Wing A and Wing B

import pandas as pd
from scipy.stats import ttest_ind

# Read the dataset
df = pd.read_csv("Luggage.csv")

# Display first five records
print("First Five Records")
print(df.head())

# Display column names
print("\nColumn Names")
print(df.columns)

# Separate Wing A delivery times
wing_a = df[df["Wing"] == "A"]["DeliveryTime"]

# Separate Wing B delivery times
wing_b = df[df["Wing"] == "B"]["DeliveryTime"]

# Find sample sizes
n_a = len(wing_a)
n_b = len(wing_b)

print("\nNumber of deliveries in Wing A =", n_a)
print("Number of deliveries in Wing B =", n_b)

# Calculate means
mean_a = wing_a.mean()
mean_b = wing_b.mean()

print("\nMean Delivery Time - Wing A =", mean_a)
print("Mean Delivery Time - Wing B =", mean_b)

# Perform Independent Two-Sample t-Test
t_statistic, p_value = ttest_ind(wing_a, wing_b)

# Level of significance
alpha = 0.05

print("\nIndependent Two-Sample t-Test Results")
print("t-statistic =", t_statistic)
```

v@sugi

```
print("p-value =", p_value)

print("\nLevel of Significance =", alpha)

# Decision
if p_value < alpha:

    print("\nReject the Null Hypothesis")

    print("Conclusion: There is a significant difference")
    print("between the mean delivery times of Wing A and Wing B.")

else:

    print("\nFail to Reject the Null Hypothesis")

    print("Conclusion: There is no significant difference")
    print("between the mean delivery times of Wing A and Wing B.")
```

Explanation of the Program

1. Import Pandas

```
import pandas as pd
```

Pandas is used for reading and processing the CSV dataset.

2. Import the t-Test

```
from scipy.stats import ttest_ind
```

`ttest_ind()` is used to perform an **Independent Two-Sample t-Test**.

3. Read the Dataset

```
df = pd.read_csv("Luggage.csv")
```

This reads the CSV file into a DataFrame.

4. Select Wing A

```
wing_a = df[df["Wing"] == "A"]["DeliveryTime"]
```

This selects only the delivery times where the wing is A.

5. Select Wing B

@suji

```
wing_b = df[df["Wing"] == "B"]["DeliveryTime"]
```

This selects only the delivery times where the wing is B.

6. Calculate the Means

```
mean_a = wing_a.mean()
```

```
mean_b = wing_b.mean()
```

These calculate the average luggage delivery time for each wing.

7. Perform the t-Test

```
t_statistic, p_value = ttest_ind(wing_a, wing_b)
```

The function compares the means of the two independent samples.
It returns:

- t-statistic
- p-value

8. Make the Decision

```
if p_value < alpha:
```

If the p-value is less than 0.05, the null hypothesis is rejected.

Output

Using your **actual attached dataset**, you should get approximately:

First Five Records

```
  delivery_id Wing  DeliveryTime

0         A01    A            12
1         A02    A            15
2         A03    A            13
3         A04    A            14
4         A05    A            16
```

Column Names

```
Index(['delivery_id', 'Wing', 'DeliveryTime'], dtype='object')

Number of deliveries in Wing A = 20
Number of deliveries in Wing B = 20

Mean Delivery Time - Wing A = 13.8
Mean Delivery Time - Wing B = 17.8
```

Independent Two-Sample t-Test Results

```
t-statistic = -7.290188827928476
```

p-value = 9.992186458071817e-09

Level of Significance = 0.05

Reject the Null Hypothesis

Conclusion: There is a significant difference between the mean delivery times of Wing A and Wing B.

Important Results

Statistic	Wing A	Wing B
Number of deliveries	20	20
Mean delivery time	**13.8 min**	**17.8 min**
Standard deviation	1.7351	1.7351

Test results:

$$t = -7.2902$$

$$p = 9.9922 \times 10^{-9}$$

or approximately:

$$p = 0.00000000999$$

Interpretation

The average delivery time for:

Wing A:

$$13.8 \text{ minutes}$$

Wing B:

$$17.8 \text{ minutes}$$

Therefore, Wing B takes approximately:

$$17.8 - 13.8 = 4$$

minutes longer on average than Wing A.

The p-value is:

$$0.00000000999$$

Since:

$$0.00000000999 < 0.05$$

we **reject the null hypothesis**.

Final Conclusion

There is **statistically significant evidence of a difference between the mean luggage delivery times of Wing A and Wing B**.

The data show that:

- Wing A average = **13.8 minutes**
- Wing B average = **17.8 minutes**

Therefore, **Wing B has a significantly higher average luggage delivery time than Wing A**.

Result

The Independent Two-Sample t-Test was successfully performed on the Luggage.csv dataset at the **5% level of significance**.

Since the p-value is less than 0.05, the null hypothesis is rejected.

Hence, there is a significant difference between the mean luggage delivery times of Wing A and Wing B.

Viva Questions – Experiment 5

Basic Questions

1. What is hypothesis testing?
2. What is an Independent Two-Sample t-Test?
3. Why is a Two-Sample t-Test used in this experiment?
4. What are the two groups being compared?
5. What is the sample size of Wing A?
6. What is the sample size of Wing B?
7. What is the mean delivery time of Wing A?
8. What is the mean delivery time of Wing B?
9. What is the level of significance?
10. Is this a one-tailed or two-tailed test? Why?

Hypothesis Questions

11. What is the null hypothesis?
12. What is the alternative hypothesis?
13. Why is the alternative hypothesis written as: H1:μA=μB
14. What does it mean to reject the null hypothesis?

Python Questions

15. Which Python library is used for the t-Test?
16. What is the purpose of `ttest_ind()`?
17. What does the following statement do?

```
wing_a = df[df["Wing"] == "A"]["DeliveryTime"]
```

18. How is the mean calculated in Python?
19. What does the p-value indicate?
20. Why do we compare the p-value with 0.05?

Output Questions

21. What is the t-statistic obtained?
22. What is the p-value obtained?
23. Is the p-value less than 0.05?
24. Should the null hypothesis be rejected?
25. Which wing has the higher average delivery time?
26. How much longer is the average delivery time in Wing B?

Most Important Viva Question

Why is this an independent two-sample t-test?

Answer: Because the experiment compares the mean delivery times of two separate and independent groups, Wing A and Wing B. We are interested in determining whether their population means are significantly different. Therefore, an Independent Two-Sample t-Test is appropriate.

v@suji

EXPERIMENT NO. 6

Title

Paired t-Test for Comparing the Compressive Strength of Concrete at 2 Days and 7 Days

Aim

To determine whether there is sufficient evidence that the **mean compressive strength of concrete at 2 days is lower than the mean compressive strength at 7 days**, at the **0.01 level of significance**.

Problem Statement

The file **Concrete.csv** contains the compressive strength, in thousands of pounds per square inch, of **40 samples of concrete** taken **2 days and 7 days after pouring**.

At the **0.01 level of significance**, determine whether there is evidence that the **mean strength is lower at 2 days than at 7 days**.

Learning Objectives

After completing this experiment, students will be able to:

- Understand hypothesis testing for paired observations.
- Understand the Paired t-Test.
- Formulate null and alternative hypotheses.
- Calculate the mean compressive strength at 2 and 7 days.
- Calculate the t-statistic and p-value using Python.
- Make a statistical decision based on the significance level.
- Interpret the result in the context of concrete strength.

Theory

The same **40 concrete samples** are measured twice:

- Once after **2 days**
- Once after **7 days**

Therefore, the observations are **paired/dependent observations**.

For this reason, a **Paired t-Test** is appropriate.

The test examines the difference between the strength measurements of each concrete sample.

Let: μ2=Mean strength at 2 days μ7=Mean strength at 7 days

The question asks whether the mean strength at 2 days is **lower** than at 7 days.

Therefore, this is a **one-tailed test (left-tailed test)**.

Hypotheses

Null Hypothesis

$$H_0 : \mu_2 \geq \mu_7$$

The mean compressive strength at 2 days is not lower than the mean strength at 7 days.

Alternative Hypothesis

$$H_1 : \mu_2 < \mu_7$$

The mean compressive strength at 2 days is lower than the mean strength at 7 days.

Level of Significance

Given:

$$\alpha = 0.01$$

Therefore, the test is conducted at the **1% level of significance**.

Formula

For a Paired t-Test, first calculate the difference for each pair:

$$d_i = X_{2i} - X_{7i}$$

Then calculate the mean difference:

$$\bar{d} = \frac{\sum d_i}{n}$$

The t-statistic is:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

where:

- $\bar{d}$ = Mean of the paired differences
- s_d = Standard deviation of the paired differences
- n = Number of pairs

For this experiment:

$$n = 40$$

Decision Rule

Compare the p-value with:

$$\alpha = 0.01$$

If:

$$p\text{-value} < 0.01$$

Reject H_0.

If:

$$p\text{-value} \geq 0.01$$

Fail to Reject H_0.

Software Requirements

- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas
- SciPy
- Concrete.csv

Dataset Description

The supplied dataset contains **40 concrete samples**.

Column	Description
sample_id	Unique identification number of the concrete sample
2_days	Compressive strength after 2 days
7_days	Compressive strength after 7 days

The strength is measured in **thousands of pounds per square inch**.

Algorithm

Step 1: Open Jupyter Notebook.

Step 2: Import Pandas and the Paired t-Test function from SciPy.

Step 3: Read the Concrete.csv dataset.

Step 4: Display the first five records.

Step 5: Display the column names.

Step 6: Select the strength values at 2 days.

Step 7: Select the strength values at 7 days.

Step 8: Calculate the mean strength at 2 days.

Step 9: Calculate the mean strength at 7 days.

Step 10: Perform the Paired t-Test.

Step 11: Obtain the t-statistic and p-value.

Step 12: Compare the p-value with 0.01.

Step 13: Make the statistical decision.

Step 14: State the final conclusion.

Procedure

Step 1: Open Jupyter Notebook

Open **Anaconda Navigator** and launch **Jupyter Notebook**.

Step 2: Create a New Notebook

Create a new Python notebook.

Step 3: Place the Dataset

Place the file: Concrete.csv

in the same folder as the Jupyter Notebook.

Step 4: Import the Required Libraries

```
import pandas as pd
from scipy.stats import ttest_rel
```

`ttest_rel()` is used for a **Paired t-Test**.

Step 5: Read the Dataset

```
df = pd.read_csv("Concrete.csv")
```

@sugi

Step 6: Display the First Five Records

```
print("First Five Records")
print(df.head())
```

Step 7: Display Column Names

```
print("\nColumn Names")
print(df.columns)
```

Step 8: Select the 2-Day Strength

```
strength_2_days = df["2_days"]
```

Step 9: Select the 7-Day Strength

```
strength_7_days = df["7_days"]
```

Step 10: Calculate the Means

```
mean_2_days = strength_2_days.mean()
mean_7_days = strength_7_days.mean()
```

Step 11: Perform the Paired t-Test

```
t_statistic, p_value = ttest_rel(
    strength_2_days,
    strength_7_days )
```

Step 12: Set the Significance Level

```
alpha = 0.01
```

Step 13: Make the Decision

```
if p_value < alpha:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

v@suji

Python Program

```
# Experiment 6
# Paired t-Test for Concrete Strength

import pandas as pd
from scipy.stats import ttest_rel

# Read the dataset
df = pd.read_csv("Concrete.csv")

# Display first five records
print("First Five Records")
print(df.head())

# Display column names
print("\nColumn Names")
print(df.columns)

# Find total number of samples
n = len(df)

print("\nTotal Number of Samples =", n)

# Select concrete strength at 2 days
strength_2_days = df["2_days"]

# Select concrete strength at 7 days
strength_7_days = df["7_days"]

# Calculate mean strength
mean_2_days = strength_2_days.mean()
mean_7_days = strength_7_days.mean()

print("\nMean Strength at 2 Days =", mean_2_days)
print("Mean Strength at 7 Days =", mean_7_days)
```

vtusuji

```
# Perform Paired t-Test
t_statistic, p_value = ttest_rel(
    strength_2_days,
    strength_7_days
)

# Level of significance
alpha = 0.01

print("\nPaired t-Test Results")
print("t-statistic =", t_statistic)
print("p-value =", p_value)

print("\nLevel of Significance =", alpha)

# Decision
if p_value < alpha:

    print("\nReject the Null Hypothesis")

    print("Conclusion: There is sufficient evidence
that")
    print("the mean concrete strength at 2 days is
lower")
    print("than the mean concrete strength at 7
days.")

else:
    print("\nFail to Reject the Null Hypothesis")

    print("Conclusion: There is not sufficient
evidence that")
    print("the mean concrete strength at 2 days is
lower")
    print("than the mean concrete strength at 7
days.")
```

Explanation of the Program

1. Import Pandas

```
import pandas as pd
```

Pandas is used to read and process the CSV dataset.

2. Import the Paired t-Test

```
from scipy.stats import ttest_rel
```

`ttest_rel()` performs a **Paired t-Test**.

3. Read the Dataset

```
df = pd.read_csv("Concrete.csv")
```

This reads the concrete dataset into a DataFrame.

4. Select 2-Day Strength

```
strength_2_days = df["2_days"]
```

This extracts the compressive strength values measured after 2 days.

5. Select 7-Day Strength

```
strength_7_days = df["7_days"]
```

This extracts the compressive strength values measured after 7 days.

6. Calculate the Means

```
mean_2_days = strength_2_days.mean()

mean_7_days = strength_7_days.mean()
```

These calculate the average strength at 2 and 7 days.

vtusuji

7. Perform the Paired t-Test

```
t_statistic, p_value = ttest_rel(strength_2_days, strength_7_days)
```

The function compares each sample's 2-day strength with its corresponding 7-day strength.

8. Set Significance Level

```
alpha = 0.01
```

The problem specifies a 1% significance level.

9. Make the Decision

```
if p_value < alpha:
```

If the p-value is less than 0.01, the null hypothesis is rejected.

Expected Output

For your **attached Concrete dataset**, the output will be approximately:

First Five Records

	sample_id	2_days	7_days
0	C01	3.46	5.32
1	C02	3.92	5.38
2	C03	3.56	5.53
3	C04	3.64	5.22
4	C05	3.24	4.74

Column Names

```
Index(['sample_id', '2_days', '7_days'], dtype='object')
```

Total Number of Samples = 40

vosuji

Mean Strength at 2 Days = 3.3915

Mean Strength at 7 Days = 5.012

Paired t-Test Results

t-statistic = -44.44990475760902

p-value = 5.02252161413093e-35

Level of Significance = 0.01

Reject the Null Hypothesis

Conclusion: There is sufficient evidence that the mean concrete strength at 2 days is lower than the mean concrete strength at 7 days.

Important Results

Statistic	Value
Number of samples	40
Mean strength at 2 days	3.3915
Mean strength at 7 days	5.0120
t-statistic	-44.4499
p-value	5.0225×10^{-35}
Significance level	0.01
Decision	Reject H_0

Interpretation

The mean compressive strength at 2 days is:

$$\bar{X}_2 = 3.3915$$

and the mean compressive strength at 7 days is:

$$\bar{X}_7 = 5.0120$$

Therefore, the average strength at 2 days is lower than the average strength at 7 days.

The calculated t-statistic is:

$$t = -44.4499$$

and the p-value is:

$$p = 5.0225 \times 10^{-35}$$

This is an extremely small p-value.

Since:

$$5.0225 \times 10^{-35} < 0.01$$

we **reject the null hypothesis**.

Final Conclusion

At the **0.01 level of significance**, there is **sufficient statistical evidence that the mean compressive strength of concrete at 2 days is lower than the mean compressive strength at 7 days.**

Therefore:

$$\boxed{\text{Reject } H_0}$$

and conclude that concrete strength is significantly higher at 7 days than at 2 days.

Result

The **Paired t-Test** was successfully performed on the Concrete.csv dataset.

At the **1% level of significance**, the p-value is less than 0.01. Hence, the null hypothesis is rejected.

Therefore, there is significant evidence that the mean compressive strength of concrete is lower at 2 days than at 7 days.

Viva Questions – Experiment 6

Basic Questions

1. What is hypothesis testing?
2. What is a Paired t-Test?
3. Why is a Paired t-Test used in this experiment?
4. How many concrete samples are present in the dataset?
5. What are the two measurements taken for each concrete sample?
6. What is the mean strength at 2 days?
7. What is the mean strength at 7 days?
8. What is the level of significance?

Hypothesis Questions

9. What is the null hypothesis?
10. What is the alternative hypothesis?
11. Is this a one-tailed or two-tailed test?
12. Why is this a one-tailed test?
13. What does H1:μ2<μ7 mean?

Python Questions

14. Which Python function is used for the Paired t-Test?
15. What is the purpose of `ttest_rel()`?
16. What does `df["2_days"]` do?
17. What does `.mean()` calculate?
18. What does the p-value represent?
19. Why is the significance level set to 0.01?
20. What happens if the p-value is less than 0.01?

Output Questions

21. What is the calculated t-statistic?
22. What is the calculated p-value?
23. Is the p-value less than 0.01?
24. Should the null hypothesis be rejected?
25. What is the final conclusion?

Most Important Viva Question

Why is a Paired t-Test used instead of an Independent Two-Sample t-Test?

Answer: The **same 40 concrete samples** are measured at both 2 days and 7 days. Therefore, each 2-day observation has a corresponding 7-day observation from the **same sample**. The observations are paired, so a **Paired t-Test** is appropriate.

v@sugi

EXPERIMENT NO. 7

Title

Chi-Square Test of Independence for Promotion Status of Employees

Aim

To determine whether there is a significant association between **company origin (Company A or Company B)** and **promotion status**, and thereby examine the claim that management favours employees who originally belonged to **Company A**, using a hypothesis test at **95% confidence level**.

Problem Statement

Two companies, **A and B**, were merged. After the first appraisal cycle following the merger, employees originally belonging to Company B alleged that the management favours employees who were originally part of Company A.

The promotion data are:

Company	Promoted (P)	Not Promoted (NP)	Total
A	15	9	24
B	16	15	31
Total	**31**	**24**	**55**

At **95% confidence**, perform a hypothesis test to determine whether the claim is supported by the data.

Learning Objectives

After completing this experiment, students will be able to:

- Understand the Chi-Square Test of Independence.
- Work with categorical data in a contingency table.
- Formulate null and alternative hypotheses.
- Calculate expected frequencies.
- Calculate the Chi-Square test statistic.
- Interpret the p-value.
- Determine whether two categorical variables are associated.
- Draw a conclusion from a statistical test using Python.

Theory

The **Chi-Square Test of Independence** is used to determine whether there is a significant association between two categorical variables.

In this experiment, the two categorical variables are:

1. **Company** – A or B
2. **Promotion Status** – Promoted (P) or Not Promoted (NP)

We want to determine whether promotion status is associated with the employee's original company.

If there is no association, promotion status and company origin are considered independent.

Hypotheses

Null Hypothesis H_0

$$H_0 :$$

There is **no significant association** between the employee's original company and promotion status.

In other words, the promotion status is independent of whether the employee originally belonged to Company A or Company B.

Alternative Hypothesis H_1

$$H_1 :$$

There is a **significant association** between the employee's original company and promotion status.

This would provide evidence that promotion outcomes are related to the employee's original company.

Level of Confidence and Significance

The question specifies **95% confidence**.

Therefore:

$$\alpha = 1 - 0.95$$

$$\boxed{\alpha = 0.05}$$

Thus, the hypothesis test is conducted at the **5% level of significance**.

Why Chi-Square Test?

Both variables are categorical:

- Company → A / B
- Promotion Status → P / NP

Therefore, a **Chi-Square Test of Independence** is appropriate.

Formula

The Chi-Square test statistic is:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

where:

- O = Observed frequency
- E = Expected frequency

The expected frequency is calculated using:

$$E = \frac{(\text{Row Total})(\text{Column Total})}{\text{Grand Total}}$$

Expected Frequencies

The observed table is:

Company	P	NP	Total
A	15	9	24
B	16	15	31
Total	**31**	**24**	**55**

Expected frequency for A-P

$$E_{AP} = \frac{24 \times 31}{55}$$

$$E_{AP} = 13.5273$$

Expected frequency for A-NP

$$E_{ANP} = \frac{24 \times 24}{55}$$

$$E_{ANP} = 10.4727$$

Expected frequency for B-P

$$E_{BP} = \frac{31 \times 31}{55}$$

$$E_{BP} = 17.4727$$

Expected frequency for B-NP

$$E_{BNP} = \frac{31 \times 24}{55}$$

$$E_{BNP} = 13.5273$$

Therefore:

Company	P	NP
A	13.5273	10.4727
B	17.4727	13.5273

Degrees of Freedom

The formula is:

$$df = (r-1)(c-1)$$

where:

- r = number of rows
- c = number of columns

Here:

$$r = 2, \quad c = 2$$

Therefore:

$$df = (2-1)(2-1)$$

$$\boxed{df = 1}$$

Decision Rule

Using the p-value approach:

If

$$p < 0.05$$

then **Reject** H_0.

If

$$p \geq 0.05$$

then **Fail to Reject** H_0.

Software Requirements

- Python 3.x
- Anaconda
- Jupyter Notebook
- Pandas
- SciPy
- Promotions.csv

Dataset Description

The supplied dataset represents the promotion status of employees from the two companies.

Variable	Description
Company	Original company of the employee
P	Employee was promoted
NP	Employee was not promoted

There are **55 employees** in total.

- Company A = 24 employees
- Company B = 31 employees
- Promoted = 31 employees
- Not Promoted = 24 employees

Algorithm

Step 1: Open Jupyter Notebook.
Step 2: Import Pandas and the Chi-Square test function from SciPy.
Step 3: Read the Promotions.csv dataset.
Step 4: Display the dataset.
Step 5: Create the observed frequency table.
Step 6: Perform the Chi-Square Test of Independence.
Step 7: Obtain the Chi-Square statistic, p-value, degrees of freedom and expected frequencies.
Step 8: Set the significance level to 0.05.
Step 9: Compare the p-value with 0.05.
Step 10: Make the statistical decision.
Step 11: Interpret the result in relation to the management favouritism claim.

Procedure

Step 1: Open Jupyter Notebook

Open **Anaconda Navigator** and launch Jupyter Notebook.

Step 2: Create a New Notebook

Create a new Python notebook.

Step 3: Place the Dataset

Place the file:

Promotions.csv

in the same folder as the Jupyter Notebook.

Step 4: Import the Required Libraries

```
import pandas as pd

from scipy.stats import chi2_contingency
```

Step 5: Read the Dataset

```
df = pd.read_csv("Promotions.csv")
```

Step 6: Display the Dataset

```
print("Promotion Data")

print(df)
```

Step 7: Create the Observed Frequency Table

The data can be represented as:

```
observed = [
    [15, 9],
    [16, 15]
]
```

The rows represent Companies A and B, while the columns represent Promoted and Not Promoted.

Step 8: Perform the Chi-Square Test

```
chi_square, p_value, degrees_of_freedom, expected =
chi2_contingency(observed,correction=False)
```

vtusuji

Step 9: Set the Significance Level

```
alpha = 0.05
```

Step 10: Make the Decision

```
if p_value < alpha:
    print("Reject the Null Hypothesis")
else:
    print("Fail to Reject the Null Hypothesis")
```

Python Program

```
# Experiment 7
# Chi-Square Test of Independence
# Promotion Status of Employees

import pandas as pd
from scipy.stats import chi2_contingency

# Read the dataset
df = pd.read_csv("Promotions.csv")

# Display the data
print("Promotion Data")
print(df)

# Create observed frequency table
observed = [
    [15, 9],
    [16, 15]
]
# Perform Chi-Square Test
chi_square, p_value, degrees_of_freedom, expected =
chi2_contingency(observed,correction=False)

# Display expected frequencies
print("\nExpected Frequencies")
print(expected)
```

```
# Display test results
print("\nChi-Square Test Results")
print("Chi-Square Statistic =", chi_square)
print("Degrees of Freedom =", degrees_of_freedom)
print("p-value =", p_value)

# Level of significance
alpha = 0.05

print("\nLevel of Significance =", alpha)

# Decision
if p_value < alpha:

    print("\nReject the Null Hypothesis")

    print("Conclusion: There is a significant
association")
    print("between company and promotion status.")

else:

    print("\nFail to Reject the Null Hypothesis")

    print("Conclusion: There is no significant
association")
    print("between company and promotion status.")
```

Explanation of the Program

1. Import Pandas

```
import pandas as pd
```

Pandas is used to read and handle the dataset.

2. Import Chi-Square Test

```
from scipy.stats import chi2_contingency
```

vigsuji

chi2_contingency() performs the Chi-Square Test of Independence.

3. Read the Dataset

```
df = pd.read_csv("Promotions.csv")
```

This reads the CSV file.

4. Create the Observed Table

```
observed = [
    [15, 9],
    [16, 15]
]
```

The table represents:

	P	NP
A	15	9
B	16	15

5. Perform the Test

```
chi_square, p_value, degrees_of_freedom, expected = chi2_contingency(observed,correction=False)
```

The function returns:

- Chi-Square statistic
- p-value
- Degrees of freedom
- Expected frequencies

6. Set Significance Level

```
alpha = 0.05
```

Because the question specifies 95% confidence: α=0.05

vaasugi

7. Make the Decision

The program compares the p-value with 0.05.

Expected Output

Promotion Data

	Company	P	NP	Total
0	A	15	9	24
1	B	16	15	31

Expected Frequencies

[[13.52727273 10.47272727]
 [17.47272727 13.52727273]]

Chi-Square Test Results

Chi-Square Statistic = 0.6519088189386055
Degrees of Freedom = 1
p-value = 0.41943105261448455

Level of Significance = 0.05

Fail to Reject the Null Hypothesis

Conclusion: There is no significant association between company and promotion status.

v@sugi

```
Promotion Data
   employee_id original_company promotion_status
0         E001                A         Promoted
1         E002                A         Promoted
2         E003                A         Promoted
3         E004                A         Promoted
4         E005                A         Promoted
5         E006                A         Promoted
6         E007                A         Promoted
7         E008                A         Promoted
8         E009                A         Promoted
9         E010                A         Promoted
10        E011                A         Promoted
11        E012                A         Promoted
12        E013                A         Promoted
13        E014                A         Promoted
14        E015                A         Promoted
15        E016                A     Not Promoted
16        E017                A     Not Promoted
17        E018                A     Not Promoted
18        E019                A     Not Promoted
19        E020                A     Not Promoted
20        E021                A     Not Promoted
21        E022                A     Not Promoted
22        E023                A     Not Promoted
23        E024                A     Not Promoted
24        E025                B         Promoted
25        E026                B         Promoted
26        E027                B         Promoted
27        E028                B         Promoted
28        E029                B         Promoted
29        E030                B         Promoted
30        E031                B         Promoted
31        E032                B         Promoted
32        E033                B         Promoted
33        E034                B         Promoted
34        E035                B         Promoted
35        E036                B         Promoted
36        E037                B         Promoted
37        E038                B         Promoted
38        E039                B         Promoted
39        E040                B         Promoted
40        E041                B     Not Promoted
41        E042                B     Not Promoted
42        E043                B     Not Promoted
43        E044                B     Not Promoted
44        E045                B     Not Promoted
45        E046                B     Not Promoted
46        E047                B     Not Promoted
47        E048                B     Not Promoted
48        E049                B     Not Promoted
49        E050                B     Not Promoted
50        E051                B     Not Promoted
51        E052                B     Not Promoted
52        E053                B     Not Promoted
53        E054                B     Not Promoted
54        E055                B     Not Promoted

Expected Frequencies
[[13.52727273 10.47272727]
 [17.47272727 13.52727273]]

Chi-Square Test Results
Chi-Square Statistic = 0.6519088189386055
Degrees of Freedom = 1
p-value = 0.41943105261448455

Level of Significance = 0.05

Fail to Reject the Null Hypothesis
Conclusion: There is no significant association
between company and promotion status.
```

Important Results

Statistic	Value
Total employees	55
Company A	24
Company B	31
Promoted	31
Not Promoted	24
Chi-Square statistic	**0.6519**
Degrees of freedom	**1**
p-value	**0.4194**
Significance level	**0.05**
Decision	**Fail to Reject H_0**

Interpretation

The calculated Chi-Square statistic is:

$$\chi^2 = 0.6519$$

The p-value is:

$$p = 0.4194$$

Compare:

$$0.4194 > 0.05$$

Therefore, we **fail to reject the null hypothesis**.

There is **not enough statistical evidence** to conclude that promotion status is significantly associated with the employee's original company.

Checking the Promotion Rates

It is also useful to look at the promotion percentages.

Company A

$$P(\text{Promotion}|A) = \frac{15}{24}$$

$$= 0.625$$

Therefore:

$$\boxed{62.5\%}$$

of Company A employees were promoted.

Company B

$$P(\text{Promotion}|B) = \frac{16}{31}$$

$$= 0.5161$$

Therefore:

$$\boxed{51.61\%}$$

of Company B employees were promoted.

Although Company A has a **higher observed promotion rate** than Company B, the Chi-Square test shows that the difference is **not statistically significant at the 5% level**.

Final Conclusion

At the **95% confidence level**:

$$p = 0.4194 > 0.05$$

Therefore, we **fail to reject** H_0.

There is **insufficient statistical evidence to validate the claim that management favours employees who originally belonged to Company A**.

Although the observed promotion rate is higher for Company A (**62.5%**) than Company B (**51.61%**), the difference is not statistically significant based on the available data.

Result

The **Chi-Square Test of Independence** was successfully performed on the promotion data.

Since:

$$p = 0.4194 > 0.05$$

the null hypothesis is **not rejected**.

Hence, at the 95% confidence level, there is insufficient evidence to conclude that promotion status depends on the employee's original company or that management significantly favours employees from Company A.

Viva Questions – Experiment 7

Basic Questions

1. What is hypothesis testing?
2. What is a Chi-Square Test?
3. What is a Chi-Square Test of Independence?
4. Why is the Chi-Square Test used in this experiment?
5. What are the two categorical variables in this experiment?
6. What is the total number of employees?
7. How many employees originally belonged to Company A?
8. How many employees originally belonged to Company B?

Hypothesis Questions

9. What is the null hypothesis?
10. What is the alternative hypothesis?
11. What is the confidence level?
12. What is the level of significance?
13. Why is the significance level 0.05 when the confidence level is 95%?

Formula Questions

14. What is the formula for the Chi-Square statistic?
15. How is expected frequency calculated?
16. What are the degrees of freedom?

17. How are degrees of freedom calculated for a contingency table?

Python Questions

18. Which Python library is used for the Chi-Square Test?
19. What is the purpose of `chi2_contingency()`?
20. What does the `p_value` represent?
21. Why is `correction=False` used in the program?
22. What happens if the p-value is less than 0.05?
23. What happens if the p-value is greater than 0.05?

Output Questions

24. What is the calculated Chi-Square statistic?
25. What is the p-value?
26. What are the degrees of freedom?
27. Should the null hypothesis be rejected?
28. What is the promotion rate of Company A?
29. What is the promotion rate of Company B?
30. Does the higher observed promotion rate of Company A automatically prove favouritism?

Answer: No. The difference in observed promotion rates must also be statistically significant. In this experiment, the p-value is **0.4194**, which is greater than 0.05, so there is insufficient evidence to establish a significant association.

Most Important Viva Question

Why is the Chi-Square Test of Independence appropriate for this problem?

Answer: Because both variables are **categorical**—the employee's original company (A or B) and promotion status (Promoted or Not Promoted). The test determines whether there is a statistically significant association between these two categorical variables.

EXPERIMENT NO. 8

Title

One-Way ANOVA – Analysis of Carbon Emissions Based on Fuel Type

Aim

To determine whether the **mean carbon emission differs significantly among different fuel types** using **One-Way ANOVA** at **5% level of significance**.

Problem Statement

A traffic management inspector in a city wants to understand whether carbon emissions from different cars are different. For this reason, the inspector has taken random samples from all registered cars on the road in that city and would like to test if the amount of carbon emission released depends on fuel type at **5% significance level**.

Dataset: AOVData.csv

The attached dataset contains the following columns:

Column	Description
car_id	Identification number of the car
FuelType	Type of fuel used by the car
CarbonEmission	Carbon emission value

Theory

What is ANOVA?

ANOVA stands for **Analysis of Variance**.

ANOVA is a statistical technique used to determine whether the means of **three or more groups** are significantly different.
In this experiment, we want to determine whether the mean carbon emission is different for different fuel types.

The groups are:

- Petrol
- Diesel
- Hybrid
- Electric

Therefore, **One-Way ANOVA** is suitable.

Variables Used

Independent Variable

FuelType

It is a categorical variable containing:

- Petrol
- Diesel
- Hybrid
- Electric

Dependent Variable

CarbonEmission

It is a numerical variable representing the carbon emission value.

v@sugi

Hypotheses

Null Hypothesis H_0

The mean carbon emissions are equal for all fuel types.

$$H_0 : \mu_{Petrol} = \mu_{Diesel} = \mu_{Hybrid} = \mu_{Electric}$$

In simple words:

> **Fuel type does not significantly affect carbon emissions.**

Alternative Hypothesis H_1

At least one fuel type has a significantly different mean carbon emission.

In simple words:

> **Fuel type has a significant effect on carbon emissions.**

Level of Significance

The question specifies:

$$\boxed{\alpha = 0.05}$$

Therefore, the test is performed at the **5% level of significance**.

Formula

The ANOVA test statistic is:

$$F = \frac{MS_B}{MS_W}$$

where:

- MS_B = Mean Square Between Groups
- MS_W = Mean Square Within Groups

The mean square values are calculated as:

$$MS_B = \frac{SS_B}{df_B}$$

and

$$MS_W = \frac{SS_W}{df_W}$$

Therefore:

$$F = \frac{SS_B/df_B}{SS_W/df_W}$$

In this experiment, Python's `f_oneway()` function calculates the ANOVA statistic automatically.

Decision Rule

The decision is made using the **p-value**.

If:

$$p < 0.05$$

Reject H_0.

If:

$$p \geq 0.05$$

Fail to Reject H_0.

Software Requirements

- Anaconda
- Jupyter Notebook
- Python
- Pandas
- SciPy
- AOVData.csv

Note: Your attached file is named carbonemissions - Carbon Emissions(1).csv, but the question calls the dataset AOVData.csv. For the lab manual, I recommend that you rename the file to **AOVData.csv** so that students can use the filename given in the question.

Algorithm

1. Start the Jupyter Notebook.
2. Import the Pandas library.
3. Import the f_oneway() function from SciPy.
4. Read the AOVData.csv file.
5. Display the first five records.
6. Display the column names.
7. Find the total number of observations.
8. Identify the different fuel types.
9. Calculate the mean carbon emission for each fuel type.
10. Separate the carbon-emission values according to fuel type.
11. Perform One-Way ANOVA.
12. Obtain the F-statistic and p-value.
13. Set the significance level to 0.05.
14. Compare the p-value with 0.05.
15. Accept or reject the null hypothesis.
16. Write the final conclusion.

Procedure

Step 1: Open Jupyter Notebook

Open **Anaconda Navigator** and launch **Jupyter Notebook**.

Step 2: Create a New Notebook

Create a new Python notebook.

Step 3: Place the Dataset

Place the dataset in the same folder as the Jupyter Notebook.

Rename the file as: AOVData.csv

Step 4: Import Required Libraries

Enter: import pandas as pd

from scipy.stats import f_oneway

Explanation

pandas is used to read and manipulate the CSV dataset.

f_oneway() is used to perform the One-Way ANOVA test.

Step 5: Read the Dataset

```
df = pd.read_csv("AOVData.csv")
```

This reads the CSV file into a DataFrame called df.

Step 6: Display the First Five Records

```
print("First Five Records")
```

```
print(df.head())
```

This displays the first five rows of the dataset.

Step 7: Display Column Names

```
print("\nColumn Names")
```

```
print(df.columns)
```

This helps the student verify the column names.

For the attached dataset, the output is:

@sujal

```
Index(['car_id', 'FuelType', 'CarbonEmission'],
dtype='object')
```

Step 8: Find Total Number of Observations

```
total_observations = len(df)

print("\nTotal Number of Observations =",
total_observations)
```

The dataset contains: 120 observations.

Step 9: Find the Fuel Types

```
print("\nFuel Types")

print(df["FuelType"].unique())
```

The dataset contains:

```
['Petrol' 'Diesel' 'Hybrid' 'Electric']
```

Step 10: Calculate Mean Carbon Emission

```
print("\nMean Carbon Emission for Each Fuel Type")

mean_values =
df.groupby("FuelType")["CarbonEmission"].mean()

print(mean_values)
```

This calculates the average carbon emission for each fuel type.

vosuji

Python Program

```
# Experiment 8
# One-Way ANOVA
# Carbon Emissions Based on Fuel Type

import pandas as pd
from scipy.stats import f_oneway

# Step 1: Read the dataset
df = pd.read_csv("AOVData.csv")

# Step 2: Display the first five records
print("First Five Records")
print(df.head())

# Step 3: Display column names
print("\nColumn Names")
print(df.columns)

# Step 4: Find total number of observations
total_observations = len(df)

print("\nTotal Number of Observations =",
    total_observations)

# Step 5: Display fuel types
print("\nFuel Types")
print(df["FuelType"].unique())

# Step 6: Calculate mean carbon emission
# for each fuel type
print("\nMean Carbon Emission for Each Fuel Type")

mean_values = df.groupby("FuelType")["CarbonEmission"].mean()

print(mean_values)

# Step 7: Create separate groups
petrol = df[df["FuelType"] == "Petrol"]["CarbonEmission"]
diesel = df[df["FuelType"] == "Diesel"]["CarbonEmission"]
hybrid = df[df["FuelType"] == "Hybrid"]["CarbonEmission"]
electric = df[df["FuelType"] == "Electric"]["CarbonEmission"]

# Step 8: Perform One-Way ANOVA
f_statistic, p_value = f_oneway(petrol,diesel,hybrid,electric)

# Step 9: Set level of significance
alpha = 0.05
```

vo@sugi

```
# Step 10: Display ANOVA results
print("\nOne-Way ANOVA Results")

print("F-statistic =", f_statistic)

print("p-value =", p_value)

print("\nLevel of Significance =", alpha)

# Step 11: Make decision
if p_value < alpha:

    print("\nReject the Null Hypothesis")

    print("Conclusion: There is sufficient evidence")
    print("that mean carbon emissions differ")
    print("according to fuel type.")

else:

    print("\nFail to Reject the Null Hypothesis")

    print("Conclusion: There is not sufficient evidence")
    print("that mean carbon emissions differ")
    print("according to fuel type.")
```

Explanation of the Code

Reading the dataset

`df = pd.read_csv("AOVData.csv")`
Reads the CSV file into Pandas.

Finding fuel types

`df["FuelType"].unique()`

Returns the different fuel categories.

The dataset contains:

- Petrol
- Diesel
- Hybrid
- Electric

@sugi

Calculating group means

```
df.groupby("FuelType")["CarbonEmission"].mean()
```

This groups the observations according to fuel type and calculates the mean carbon emission for each group.

Creating groups

```
petrol = df[df["FuelType"] ==
"Petrol"]["CarbonEmission"]
```

This selects only the carbon-emission values of Petrol cars.

The same procedure is used for Diesel, Hybrid and Electric.

Performing ANOVA

```
f_statistic, p_value = f_oneway(
    petrol,
    diesel,
    hybrid,
    electric
)
```

This compares the four group means.

The function produces:

- F-statistic
- p-value

v@sugi

Output

Using your **actual attached dataset**, the output will be approximately:

First Five Records

```
  car_id FuelType  CarbonEmission
0  CAR001   Petrol           177.8
1  CAR002   Petrol           198.8
2  CAR003   Petrol           197.8
3  CAR004   Petrol           172.7
4  CAR005   Petrol           175.9
```

Column Names

```
Index(['car_id', 'FuelType', 'CarbonEmission'],
dtype='object')
```

Total Number of Observations = 120

Fuel Types

```
['Petrol' 'Diesel' 'Hybrid' 'Electric']
```

Mean Carbon Emission for Each Fuel Type

```
FuelType
Diesel      164.246667
Electric      4.383333
Hybrid      109.723333
Petrol      180.683333
Name: CarbonEmission, dtype: float64
```

One-Way ANOVA Results

```
F-statistic = 1851.918467024794
p-value = 9.026083821106695e-98
```

Level of Significance = 0.05

Reject the Null Hypothesis

Conclusion: There is sufficient evidence that mean carbon emissions differ according to fuel type.

Important Results

Fuel Type	Number of Cars	Mean Carbon Emission
Petrol	30	180.6833
Diesel	30	164.2467
Hybrid	30	109.7233
Electric	30	4.3833

ANOVA Results

Statistic	Value
F-statistic	**1851.9185**
p-value	**9.0261×10^{-98}**
Significance level	**0.05**
Decision	**Reject H_0**

Interpretation

We obtained:

$$p = 9.0261 \times 10^{-98}$$

This is much smaller than:

$$\alpha = 0.05$$

Therefore:

$$p < 0.05$$

Hence, we **reject the null hypothesis**.

There is sufficient statistical evidence to conclude that the mean carbon emissions **differ significantly according to fuel type**.

Comparison of Mean Carbon Emissions

From the dataset:

Petrol

$$\bar{x} = 180.6833$$

Diesel

$$\bar{x} = 164.2467$$

Hybrid

$$\bar{x} = 109.7233$$

Electric

$$\bar{x} = 4.3833$$

The means are clearly different.

However, the **ANOVA test** is what establishes whether the observed differences are statistically significant.

Final Conclusion

Since:

$$p = 9.0261 \times 10^{-98} < 0.05$$

we **reject** H_0.

Therefore:

> **There is sufficient statistical evidence at the 5% level of significance to conclude that mean carbon emissions differ significantly among the different fuel types.**

Thus, based on this dataset, **carbon emission is significantly associated with fuel type**.

Result

The One-Way ANOVA test was successfully performed.

$$\boxed{F = 1851.9185}$$

$$\boxed{p \approx 9.0261 \times 10^{-98}}$$

Since:

$$p < 0.05$$

the null hypothesis is rejected.

Hence, there is significant evidence that the mean carbon emission differs among the different fuel types.

Viva Questions – Experiment 8

Basic Questions

1. What does ANOVA stand for?
2. What is the purpose of ANOVA?
3. Why is One-Way ANOVA used in this experiment?
4. What is the independent variable?

Answer: Fuel Type.

5. What is the dependent variable?

Answer: Carbon Emission.

6. What are the fuel types present in the dataset?

Answer: Petrol, Diesel, Hybrid and Electric.

7. How many observations are present in the dataset?

Answer: 120.

8. How many observations are present for each fuel type?

Answer: 30.

Hypothesis Questions

9. What is the null hypothesis?

Answer: The mean carbon emissions are equal for all fuel types.

10. What is the alternative hypothesis?

Answer: At least one fuel type has a different mean carbon emission.

11. What is the significance level?

Answer: 0.05.

12. What is the confidence level corresponding to a 5% significance level?

Answer: 95%.

Formula Questions

13. What is the formula for the F-statistic?

$$F = \frac{MS_B}{MS_W}$$

14. What is MS_B?

Answer: Mean Square Between Groups.

15. What is MS_W?

Answer: Mean Square Within Groups.

16. What does the F-statistic measure?

Answer: It compares the variation between groups with the variation within groups.

Python Questions

17. Which library is used for One-Way ANOVA?

Answer: SciPy.

18. Which function is used?

Answer:

```python
f_oneway()
```

19. What does `groupby()` do?

Answer: It groups the dataset according to a specified categorical variable.

20. What does `.mean()` do?

Answer: It calculates the arithmetic mean.

21. What does `.unique()` do?

Answer: It returns the unique values in a column.

Output Questions

22. What is the mean carbon emission for Petrol?

Answer: Approximately **180.6833**.

23. What is the mean carbon emission for Diesel?

Answer: Approximately **164.2467**.

24. What is the mean carbon emission for Hybrid?

Answer: Approximately **109.7233**.

25. What is the mean carbon emission for Electric?

Answer: Approximately **4.3833**.

26. What is the calculated F-statistic?

Answer: Approximately **1851.9185**.

27. What is the p-value?

Answer: Approximately **9.0261×10^{-98}**.

28. Is the p-value less than 0.05?

Answer: Yes.

29. Should the null hypothesis be rejected?

Answer: Yes.

30. What is the final conclusion?

Answer: There is sufficient evidence that the mean carbon emissions differ significantly according to fuel type.

⭐ Most Important Viva Question

Why do we reject the null hypothesis in this experiment?

Answer:

The p-value is:

$$9.0261 \times 10^{-98}$$

which is much smaller than the significance level:

$$0.05$$

Therefore:

$$p < 0.05$$

Hence, we reject the null hypothesis and conclude that **there is a significant difference in mean carbon emissions among the different fuel types**.

EXPERIMENT NO. 9

Title

Correlation Analysis – Finding the Relationship Between Laptop Price and Other Factors

Aim

To find the relationship between the **price of a laptop and other factors** in the dataset laptops.csv using **Correlation Analysis**.

Problem Statement

Find the relationship between the price of a laptop with other factors of the dataset **laptops.csv**.

Theory

What is Correlation?

Correlation is a statistical technique used to measure the **strength and direction of the relationship between two numerical variables**.

For this experiment, we are interested in finding how the **Price** of a laptop is related to other numerical factors in the dataset.

For example, if the dataset contains factors such as RAM, storage, processor speed, screen size, weight, etc., we can determine whether these factors have a relationship with laptop price.

Correlation Coefficient

The correlation coefficient is represented by:

$$r$$

The value of r lies between:

$$-1 \leq r \leq +1$$

Interpretation

Correlation value	Interpretation
+1	Perfect positive correlation
0	No linear correlation
-1	Perfect negative correlation

Positive Correlation

If:

$$r > 0$$

the two variables tend to increase together.

For example, if RAM increases and laptop price also tends to increase, they have a positive correlation.

Negative Correlation

If:

$$r < 0$$

one variable tends to increase while the other tends to decrease.

Correlation close to zero

If:

$$r \approx 0$$

there is little or no linear relationship between the two variables.

Pearson Correlation Formula

The Pearson correlation coefficient is calculated using:

$$r = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sqrt{\sum(x - \bar{x})^2 \sum(y - \bar{y})^2}}$$

where:

- x = values of the first variable
- y = values of the second variable
- $\bar{x}$ = mean of the first variable
- $\bar{y}$ = mean of the second variable
- r = correlation coefficient

In this experiment, Python will calculate the correlation automatically using Pandas.

Objective of the Analysis

The students have to:

1. Read the laptop dataset.
2. Identify the numerical columns.
3. Identify the Price column.
4. Calculate the correlation between Price and other numerical factors.
5. Display the correlation values.
6. Identify which factor has the strongest positive relationship with price.
7. Identify whether any factor has a negative relationship with price.
8. Interpret the results.

Software Requirements

- Anaconda
- Jupyter Notebook
- Python
- Pandas
- Matplotlib
- laptops.csv

Algorithm

1. Start Jupyter Notebook.
2. Import the Pandas library.
3. Import Matplotlib for visualization.
4. Read the laptops.csv dataset.
5. Display the first five records.
6. Display the column names.
7. Find the total number of observations.
8. Identify the numerical columns.
9. Select the Price column.
10. Calculate the correlation matrix.
11. Extract the correlation values related to Price.
12. Arrange the correlation values in descending order.
13. Display the correlation results.
14. Plot a correlation graph if required.
15. Identify the strongest relationship with price.
16. Interpret the result.
17. Write the conclusion.

Procedure

Step 1: Open Jupyter Notebook

Open **Anaconda Navigator** and launch **Jupyter Notebook**.

Step 2: Create a New Notebook

Create a new Python notebook.

Step 3: Place the Dataset

Place: laptops.csv

in the same folder as the Jupyter Notebook.

Step 4: Import Pandas

Enter: import pandas as pd

Explanation

Pandas is used to:

- read the CSV file,
- examine the dataset,
- select columns,
- calculate correlations.

Step 5: Import Matplotlib

```
import matplotlib.pyplot as plt
```

Matplotlib can be used to display the relationship graphically.

Step 6: Read the Dataset

```
df = pd.read_csv("laptops.csv")
```

This reads the CSV file and stores it in the DataFrame `df`.

Step 7: Display the First Five Records

```
print("First Five Records")

print(df.head())
```

This helps the student understand the structure of the dataset.

Step 8: Display Column Names

```
print("\nColumn Names")

print(df.columns)
```

Students should carefully check which column represents the laptop price.

Step 9: Find Number of Observations

```
total_observations = len(df)

print("\nTotal Number of Observations =",

      total_observations)
```

This displays the number of laptop records in the dataset.

Step 10: Identify Numerical Columns

```
numeric_columns = df.select_dtypes(

    include="number").columns

print("\nNumerical Columns")

print(numeric_columns)
```

Explanation

Some laptop attributes may be text/categorical variables, while others may contain numerical values.

Correlation can be calculated between **numerical variables**.

Therefore, we first identify the numerical columns.

Step 11: Calculate Correlation Matrix

```
correlation_matrix = df[numeric_columns].corr()

print("\nCorrelation Matrix")

print(correlation_matrix)
```

The `.corr()` function calculates the Pearson correlation coefficient between the numerical variables.

vosuji

Step 12: Find Correlation with Price

If the price column is named Price, use:

```
price_correlation =
correlation_matrix["Price"].sort_values(
    ascending=False)

print("\nCorrelation with Laptop Price")

print(price_correlation)
```

This gives the correlation of every numerical factor with laptop price.

Python Program

```
# Experiment 9
# Correlation Analysis
# Relationship Between Laptop Price and Other Factors

# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt

# Step 1: Read the dataset
df = pd.read_csv("laptops.csv")

# Step 2: Display the first five records
print("First Five Records")
print(df.head())

# Step 3: Display column names
print("\nColumn Names")
print(df.columns)

# Step 4: Find total number of observations
total_observations = len(df)

print("\nTotal Number of Observations =", total_observations)




# Step 5: Identify numerical columns
```

@sugi

```
numeric_columns = df.select_dtypes(include="number").columns

print("\nNumerical Columns")
print(numeric_columns)

# Step 6: Calculate correlation matrix
correlation_matrix = df[numeric_columns].corr()

print("\nCorrelation Matrix")
print(correlation_matrix)

# Step 7: Find correlation of all numerical
# factors with Price
price_correlation = correlation_matrix["Price"].sort_values(ascending=False)

print("\nCorrelation with Laptop Price")
print(price_correlation)

# Step 8: Display the strongest relationships
print("\nFactors Related to Laptop Price")

for factor, correlation in price_correlation.items():
    print(factor, "=", round(correlation, 4))

# Step 9: Find strongest positive relationship
other_factors = price_correlation.drop("Price")

strongest_factor = other_factors.abs().idxmax()
strongest_correlation = other_factors[strongest_factor]

print("\nStrongest Relationship with Price:")
print(strongest_factor,"=",round(strongest_correlation, 4))

# Step 10: Plot correlation values
plt.figure(figsize=(10, 6))

other_factors.sort_values().plot(kind="barh")

plt.xlabel("Correlation Coefficient")
plt.ylabel("Factors")
plt.title("Correlation of Laptop Price with Other Factors")

plt.show()
```

©sugi

Explanation of Code

Reading the CSV file

```
df = pd.read_csv("laptops.csv")
```

Reads the laptop dataset.

Selecting numerical columns

```
df.select_dtypes(include="number")
```

Selects columns containing numerical data.

This is important because correlation is calculated between numerical variables.

Calculating correlation

```
df[numeric_columns].corr()
```

Calculates the Pearson correlation coefficient for the numerical columns.

Finding Price correlations

```
correlation_matrix["Price"]
```

Selects the column containing the correlation of every numerical variable with `Price`.

Sorting the correlations

```
.sort_values(ascending=False)
```

Arranges the correlation values from highest to lowest.

This makes it easier to identify which factor has the strongest positive relationship with price.

vosuji

```
First Five Records
  LaptopID Company   TypeName       CPU               GPU  RAM  StorageGB  \
0     L001   Apple   Notebook  Intel i5   NVIDIA RTX 3050    8       1024
1     L002   Apple     Gaming  Intel i3  Apple Integrated   32        512
2     L003    Dell   Notebook  Intel i3   NVIDIA RTX 4060   16       1024
3     L004      HP  Ultrabook  Intel i5   NVIDIA RTX 4060    8        512
4     L005    Acer  Ultrabook  Apple M2   NVIDIA RTX 4060   32        256

   ScreenSize  Weight    Price
0        16.0    2.08  1986.21
1        17.3    2.47  2468.10
2        14.0    1.73  1917.66
3        14.0    1.17  1681.93
4        17.3    1.36  2336.39

Column Names
Index(['LaptopID', 'Company', 'TypeName', 'CPU', 'GPU', 'RAM', 'StorageGB',
       'ScreenSize', 'Weight', 'Price'],
      dtype='object')

Total Number of Observations = 150

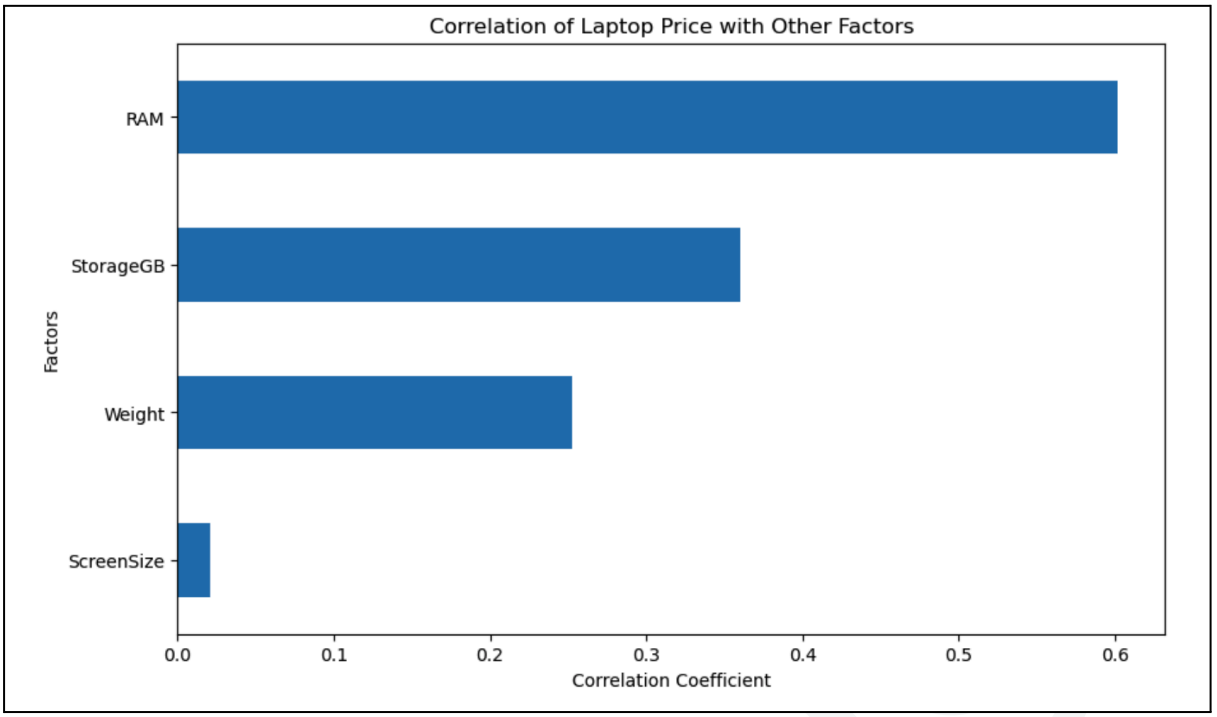
Numerical Columns
Index(['RAM', 'StorageGB', 'ScreenSize', 'Weight', 'Price'], dtype='object')
```

```
Correlation Matrix
                RAM  StorageGB  ScreenSize    Weight     Price
RAM        1.000000   0.078226    0.010423  0.007111  0.602094
StorageGB  0.078226   1.000000    0.008609 -0.121992  0.360091
ScreenSize 0.010423   0.008609    1.000000 -0.034354  0.021182
Weight     0.007111  -0.121992   -0.034354  1.000000  0.252595
Price      0.602094   0.360091    0.021182  0.252595  1.000000

Correlation with Laptop Price
Price         1.000000
RAM           0.602094
StorageGB     0.360091
Weight        0.252595
ScreenSize    0.021182
Name: Price, dtype: float64

Factors Related to Laptop Price
Price = 1.0
RAM = 0.6021
StorageGB = 0.3601
Weight = 0.2526
ScreenSize = 0.0212

Strongest Relationship with Price:
RAM = 0.6021
```



Expected Output

When the students run the program, they will get an output similar to:

First Five Records

```
        ..............
```

Column Names

```
Index([...], dtype='object')
```

```
Total Number of Observations = ...
```

Numerical Columns

```
Index([...], dtype='object')
```

Correlation Matrix

	.....
Price	**1.0000**
.....	.....
.....	.....

Correlation with Laptop Price

Price	**1.0000**
Factor 1	0.xxxx
Factor 2	0.xxxx
Factor 3	0.xxxx
Factor 4	-0.xxxx
...	

Factors Related to Laptop Price

Price = 1.0

Factor 1 = ...

Factor 2 = ...

...

Strongest Relationship with Price:

Factor X = ...

The exact values depend on your laptops.csv file.

v@sugi

Interpretation of Correlation

Students should use the following guide when interpreting their output:

Absolute value of r	Strength
0.00 – 0.19	Very weak
0.20 – 0.39	Weak
0.40 – 0.59	Moderate
0.60 – 0.79	Strong
0.80 – 1.00	Very strong

For example, if the output gives:

$$r = 0.85$$

the relationship is **very strong positive**.

If:

$$r = -0.70$$

the relationship is **strong negative**.

If:

$$r = 0.05$$

the relationship is **very weak**.

Important Note

A **correlation** does not necessarily mean causation. For **example**, if RAM and Price have a high positive correlation, it means that higher RAM values tend to be associated with higher prices in this dataset. It does not by itself prove that increasing RAM causes the price to increase.

Result

The correlation analysis is performed to determine the relationship between laptop price and the other numerical factors in laptops.csv.

The factor having the highest absolute correlation coefficient with Price is identified as having the strongest linear relationship with laptop price. The direction of the relationship is determined by the sign of the correlation coefficient:

- Positive value → positive relationship
- Negative value → negative relationship
- Value close to zero → weak/no linear relationship

Viva Questions – Experiment 9

1. What is correlation?

Correlation is a statistical measure used to determine the strength and direction of the relationship between two variables.

2. What is the range of the correlation coefficient?

$$-1 \leq r \leq 1$$

3. What does r=+1 mean?

It represents a perfect positive correlation.

4. What does r=-1 mean?

It represents a perfect negative correlation.

5. What does r=0 mean?

It indicates no linear correlation.

6. Which correlation method is used in this experiment?

Pearson correlation.

7. Which Python function calculates correlation in Pandas?

corr()

8. Why do we select numerical columns?

vasugi

Because Pearson correlation requires numerical data.

9. Why are we particularly interested in the `Price` column?

Because the objective is to determine how laptop price is related to the other factors.

10. What does a positive correlation mean?

As one variable increases, the other variable tends to increase.

11. What does a negative correlation mean?

As one variable increases, the other variable tends to decrease.

12. Does correlation prove causation?

No. Correlation only indicates the strength and direction of a relationship.

13. What is the purpose of `.sort_values()`?

It arranges the correlation values in ascending or descending order.

14. What does `select_dtypes(include="number")` do?

It selects the numerical columns from the DataFrame.

15. What is the strongest possible correlation?

$$|r|=1$$

16. How do you identify the factor most strongly related to price?

Compare the absolute values of the correlation coefficients with **`Price`**. The largest absolute value represents the strongest linear relationship.

Part-B

For **Part-B**,we recommend using **one common Excel workbook throughout all 7 experiments** rather than using different dataset for every question. This makes the lab much more practical because you can progressively build one Power BI report from data loading → cleaning → combining → visualization → interactivity → navigation.

Download the Power BI Part-B Lab Dataset by clicking the link

https://docs.google.com/spreadsheets/d/1ZPXIRgIMDupfneYKkZPMab69dqOnemDz/edit?usp=sharing&ouid=105381502433844758930&rtpof=true&sd=true

This dataset is deliberately designed so that **Experiments 1–7 can all be performed using the same workbook**.

It contains these sheets:

- Sales_2025
- Sales_2026
- Products
- Customers
- Stores
- Targets

v@sugi

POWER BI DATA ANALYTICS LAB MANUAL

Common Dataset

Excel File: PowerBI_Part_B_Lab_Dataset.xlsx

The workbook contains multiple related tables:

Sheet	Purpose
Sales_2025	Sales transactions for 2025
Sales_2026	Sales transactions for 2026
Products	Product details
Customers	Customer details
Stores	Store/location details
Targets	State-wise sales targets

This is useful because you can practice **loading, cleaning, appending, merging, visualizing, filtering, mapping, drill-through and bookmarks** using the same business scenario.

Microsoft's current Power BI documentation also recommends using **Get Data → Excel** to connect an Excel workbook and **Transform Data** to open Power Query Editor. (Microsoft Learn)

EXPERIMENT – 1

Title

Installing Power BI Desktop, Exploring the Interface and Loading Data from an Excel File

Aim: To install Power BI Desktop, explore its basic interface and load data from an Excel workbook.

Problem Statement

Install Power BI Desktop, explore the interface, and load data from an Excel file.

Theory

Power BI Desktop is a Microsoft application used to connect to data sources, transform data, create data models and build interactive reports.

Power BI Desktop primarily provides:

1. **Report View**
2. **Data/Table View**
3. **Model View**

Power Query Editor is used for preparing and transforming data before it is loaded into the report model. (Microsoft Learn)

Software Requirements

- Windows computer
- Power BI Desktop
- Microsoft Excel
- PowerBI_Part_B_Lab_Dataset.xlsx

Procedure

Step 1 – Install Power BI Desktop

1. Open a web browser.
2. Go to the official Microsoft Power BI download page.
3. Download **Power BI Desktop**.
4. Install the application.
5. Launch Power BI Desktop.

Microsoft currently provides Power BI Desktop through the Microsoft Store or Microsoft Download Center. (Microsoft Learn)

Step 2 – Explore the Interface

After opening Power BI Desktop, identify:

- Home ribbon
- Report View
- Table/Data View
- Model View
- Visualizations pane
- Data/Fields pane
- Filters pane
- Report canvas

Power BI Desktop currently provides Report, Table and Model views for working with reports and data models. (Microsoft Learn)

Step 3 – Load Excel Data

1. Open Power BI Desktop.
2. Select: **Home → Get Data → Excel Workbook**
3. Select: PowerBI_Part_B_Lab_Dataset.xlsx
4. Click **Open**.
5. The Navigator window will appear.
6. Observe the available sheets.
7. Select Sales_2025.
8. Click **Load**.

Power BI's Excel connection allows the student to select worksheet/table data from the Navigator and either **Load** it directly or choose **Transform Data** first. (Microsoft Learn)

Expected Result

The Sales_2025 table will appear in Power BI.

Students should be able to see fields such as:

- OrderID
- OrderDate
- ProductID
- CustomerID
- StoreID
- Quantity
- UnitPrice
- SalesAmount
- PaymentMode

Result

Thus, Power BI Desktop was installed successfully, its interface was explored, and data from an Excel workbook was successfully loaded.

EXPERIMENT – 2

Title

Cleaning and Transforming Data Using Power Query Editor

Aim

To clean and transform the dataset using Power Query Editor.

Problem Statement

Use Power Query Editor to clean and transform a dataset.

Theory

Power Query is the data preparation component of Power BI.

It allows users to:

- remove unnecessary columns,
- rename columns,
- change data types,
- filter rows,
- remove duplicates,
- replace values,
- create calculated/custom columns,
- group data.

Microsoft describes these operations as common query tasks in Power Query Editor. (Microsoft Learn)

Procedure

Step 1: Open the Power BI file from Experiment 1.

Step 2 : Select: **Home → Transform Data**

Power Query Editor opens.

Step 3 – Examine the Data : Select Sales_2025 from the Queries pane.

Observe:

- column names
- data types
- missing values
- duplicate records

Step 4 – Change Data Types: Check the following:

Column	Required Type
OrderID	Text
OrderDate	Date
ProductID	Text
CustomerID	Text
StoreID	Text
Quantity	Whole Number
UnitPrice	Decimal Number
SalesAmount	Decimal Number
PaymentMode	Text

To change a data type:

1. Select the column.
2. Click the data type icon.
3. Select the required type.

Correct data types are important because Power BI may automatically detect types, but the detection should be checked by the student. (Microsoft Learn)

Step 5 – Rename a Column

1. Right-click SalesAmount.
2. Select **Rename**.
3. Rename it as: Total Sales

Step 6 – Filter Data

1. Select the PaymentMode dropdown.
2. Observe the available payment methods.
3. Select only the required values.
4. Click **OK**.

Step 7 – Remove an Unnecessary Column

Yosuji

Select a column that is not required for the particular analysis.

Right-click → **Remove**.

Step 8 – Create a Custom Column

Go to: **Add Column → Custom Column**

Create:OrderValue

Formula: [Quantity] * [UnitPrice]

Step 9 – Observe Applied Steps

Look at the **Applied Steps** section.Students should observe that Power Query records each transformation.

Step 10 – Load the Data

Click: **Home → Close & Apply**

The transformed data is loaded into Power BI.

Result

The dataset was successfully cleaned and transformed using Power Query Editor.

EXPERIMENT – 3

Title

Combining Data from Multiple Tables Using Power Query

Aim

To combine data from multiple tables using **Merge** and **Append** operations in Power Query.

Problem Statement

Use Power Query Editor to combine data from multiple tables.

Theory

There are two important ways to combine data in Power Query:

Append

Append adds rows from one table below another table.

For example:

Sales_2025

+

Sales_2026

↓

All_Sales

Microsoft states that Append combines the contents of tables into a single table. (Microsoft Learn)

Merge

Merge combines columns from related tables using a common column.

For example: Sales + Products using: ProductID

Procedure – Part A: Append

Yo@sugi

Step 1 Open: **Transform Data**

Step 2 Select:**Home → Append Queries → Append Queries as New**

Step 3

Select:

- Sales_2025
- Sales_2026

Step 4 Click **OK**.

Step 5 Rename the new query: All_Sales

Procedure – Part B: Merge

Step 1: Select All_Sales.

Step 2: Select: **Home → Merge Queries → Merge Queries as New**

Step 3: Select:Products

Step 4: Select the common field: ProductID in both tables.

Step 5: Select the appropriate join.

Step 6: Click **OK**.

Step 7 : An additional column containing the matching Product table will appear.

Step 8 : Click the expand icon.

Select:

- ProductName
- Category
- UnitPrice

Step 9: Click **OK**.

Now the sales data contains product information.

Result: The 2025 and 2026 sales tables were successfully appended and the resulting table was merged with the Products table.

EXPERIMENT – 4

Title

Creating Basic Visualizations Using Bar Chart, Column Chart and Pie Chart

Aim

To visualize sales data using basic chart types in Power BI.

Problem Statement

Visualize data using basic chart types – **Bar Chart, Column Chart and Pie Chart** by loading a dataset.

v@sugi

Procedure

Load the All_Sales table created in Experiment 3.

A. Column Chart

Objective

Display total sales by product category.

Steps

1. Go to **Report View**.
2. Select **Clustered Column Chart**.
3. Drag: Category to **X-axis**.
4. Drag: Total Sales to **Y-axis / Y-axis Values**.
5. Observe the chart.

Interpretation

The column chart compares sales across product categories.

B. Bar Chart

Objective

Display sales by product.

Steps

1. Select **Bar Chart**.
2. Drag: ProductName to the category/axis field.
3. Drag: Total Sales to the value field.

The chart displays the products according to their sales.

v@sugi

C. Pie Chart

Objective

Display the contribution of different payment modes to sales.

Steps

1. Select **Pie Chart**.
2. Drag: PaymentMode to **Legend**.
3. Drag: Total Sales to **Values**.

The pie chart displays the percentage contribution of each payment mode.

Result : Bar, column and pie charts were successfully created to visualize the sales data.

EXPERIMENT – 5

Title

Creating Cards, Maps and Matrices in Power BI

Aim

To create advanced visualizations using **Cards, Maps and Matrix** visuals.

Problem Statement

Use cards, maps and matrices for advanced visuals by loading a dataset.

A. Card Visual

A Card displays a single important value.

Objective

Display total sales.

Steps

1. Select **Card**.
2. Drag: Total Sales to the card.

The card displays the total sales amount.

Create another Card

Create a card for: Quantity

This displays total quantity sold.

B. Map Visual

The dataset contains city and state information.

Steps

1. Select a Map visual.
2. Drag State or City to the **Location** field.
3. Drag Total Sales to the appropriate size/value field.
4. Observe the sales distribution geographically.

Power BI supports map visuals and other report visuals; the exact visual names/options can vary slightly by the installed Power BI Desktop version. (Microsoft Learn)

C. Matrix Visual

Objective

Display sales by state and product category.

Steps

1. Select **Matrix**.
2. Drag: State to **Rows**.
3. Drag: Category to **Columns**.
4. Drag: Total Sales to **Values**.

The matrix displays sales by state and category.

Result

Card, map and matrix visualizations were successfully created.

EXPERIMENT – 6

Title

Enhancing Dashboard Interactivity Using Slicers and Filters

Aim

To improve dashboard interactivity using slicers and filters.

Problem Statement

Enhance dashboard interactivity using slicers and filters.

v@sugi

Theory

A **Slicer** is a visual filter that allows the user to interactively select values.

Power BI also provides:

- Visual-level filters
- Page-level filters
- Report-level filters
- Drillthrough filters. (Microsoft Learn)

Procedure – Slicer

Step 1: Open the report page.

Step 2: Select **Slicer**.

Step 3: Drag: Category into the slicer.

Step 4: Select: Electronics from the slicer.

Observe that the other visuals respond to the selection.

Create Second Slicer

Create another slicer using: State

Students can now select a state and observe the change in:

- Total Sales
- Product sales
- Payment mode
- Category
- Maps

vasugi

Using Filters

Visual-Level Filter

1. Select a visual.
2. Open the Filters pane.
3. Add Category.
4. Select the desired category.

Only that visual is affected.

Page-Level Filter

1. Open Filters pane.
2. Add State under Page filters.
3. Select a state.

All visuals on that page are affected.

Report-Level Filter

Add a field under Report filters.

This applies the filter across the report.

Microsoft distinguishes visual, page, report and drillthrough filters in Power BI. (Microsoft Learn)

Result

Dashboard interactivity was successfully enhanced using slicers and filters.

vosuji

EXPERIMENT – 7

Title

Using Drill-Through Pages and Bookmarks for Navigation and Detail Views

Aim

To create drill-through pages and bookmarks for navigation and detailed analysis.

Problem Statement

Use drill-through pages and bookmarks for navigation and detail views.

Part A – Drill-Through

Objective

Create a detailed Product page that users can reach from the main dashboard.

Step 1 – Create a New Page

Click: **+ New Page**

Rename it: **Product Details**

Step 2 – Add Drill-Through Field

In the Product Details page:

1. Find the Drill-through field area.
2. Drag: ProductName into the drill-through field.

Yogesh

Step 3 – Add Visuals

Create:

- Card → Total Sales
- Card → Quantity
- Column Chart → Sales by Month
- Table → Order details

Step 4 – Return Button

Add a **Back** button to return to the main report page.

Step 5 – Test Drill-Through

Go to the main sales chart.

Right-click a product.

Select: **Drill through → Product Details**

The Product Details page will display information for the selected product.

Drillthrough is specifically designed to allow users to move from summary information to a detailed report page focused on a selected entity. (Microsoft Learn)

Part B – Bookmarks

Objective

Create different report views using bookmarks.

For example:

- **Sales Overview**
- **Product Analysis**
- **Regional Analysis**

Step 1: Create a report page containing several visuals.

Step 2: Open: **View → Bookmarks**

Step 3: Arrange the report in the desired state.

Step 4: Select: **Add Bookmark**

Rename it: Sales Overview

Step 5: Change the visible visuals or slicer selections.

Create another bookmark: Product Analysis

Step 6: Create a third bookmark: Regional Analysis

Step 7 – Add Buttons

Add buttons to the report.

Assign each button to the appropriate bookmark.

Now the user can click: **Sales Overview → Product Analysis → Regional Analysis**

to move between different report views.

Bookmarks can save the current page state, including filters, slicer selections, visual visibility and other report states. (Microsoft Learn)

RESULT

The report was successfully enhanced using:

- Drill-through pages
- Bookmarks
- Buttons
- Interactive navigation

Recommended Final Dashboard

You will be able to make or finish **one complete dashboard** after Experiment 7.

Page 1 – Sales Overview

Include:

Cards

- Total Sales
- Total Quantity
- Number of Orders

Charts

- Sales by Category – Column Chart
- Sales by Product – Bar Chart
- Sales by Payment Mode – Pie Chart

Map

- Sales by State

Slicers

- Year
- State
- Category
- Payment Mode

Page 2 – Product Analysis

Include:

- Product-wise sales
- Category-wise sales
- Quantity sold
- Product table
- Product slicer

Page 3 – Regional Analysis

v@sugi

Include:

- State-wise sales
- City-wise sales
- Map
- Matrix of State × Category
- State slicer

Page 4 – Product Details

Use: **Drill-through → ProductName**

Show:

- Product name
- Category
- Total sales
- Quantity sold
- Customer details
- Store details
- Order information

This dataset is recommended because

The workbook created is intentionally structured for the entire Part-B:

Experiment	Dataset/Sheet to use
1. Load Excel	Sales_2025
2. Clean/Transform	Sales_2025
3. Combine Tables	Sales_2025 + Sales_2026 + Products + Customers
4. Basic Charts	All_Sales
5. Cards/Maps/Matrix	All_Sales + Customers/Stores
6. Slicers/Filters	Complete report
7.Drill-through/Bookmarks	Complete report

vidsuji

One practical note: Power BI's interface is updated regularly, so the exact location or name of a visual/pane can look slightly different between versions, while the underlying workflow remains the same. Microsoft also notes this for its current Excel/Power BI tutorials. (Microsoft Learn)

v@sugi